MYAAPTHA • INTERNAL DOCUMENTATION

Corporate Project Handbook

Architecture · Data · APIs · Workflows · Roles · Operations

Source snapshot 20 October 2026

Implementation-grounded engineering and business handover

01 Executive overview and document control

MyAaptha | Corporate project handbook

Version 1.4 · Source snapshot 20 October 2026 · Internal engineering and business handover

MyAaptha combines relationship networks, private personal records, community communication and assisted organization in a shared account workspace. Browser and Flutter clients use the same Spring Boot API and PostgreSQL persistence. A separate Python service supplies deterministic ranking, duplicate detection and reviewable graph suggestions; selected backend features can also use configured external AI providers.

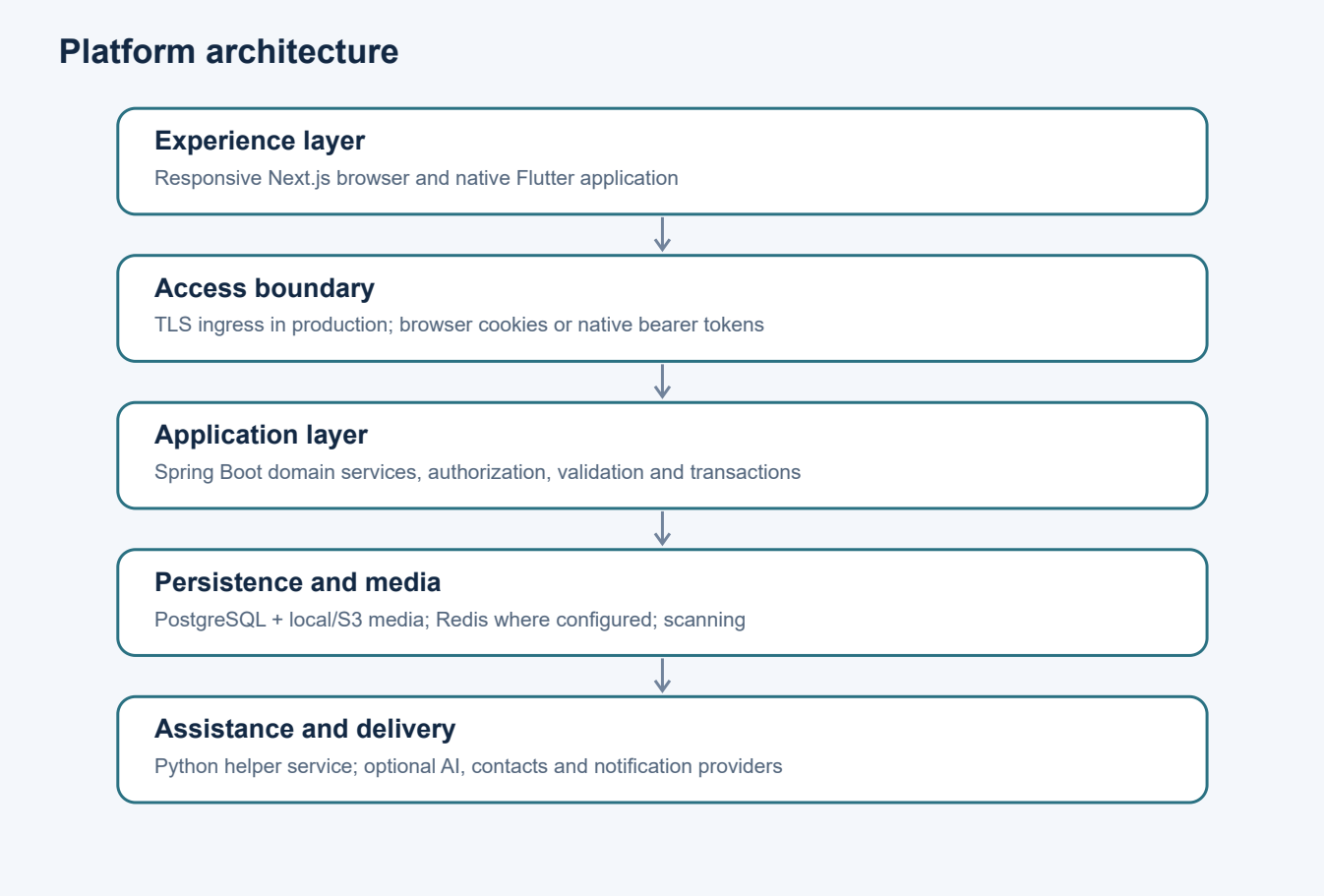
This package is intended for product owners, engineers, QA, security reviewers and operations teams. It describes the active implementation, identifies dependencies and provides editable diagrams and repeatable inventories. It does not certify a production deployment or legal compliance.

Control	Value
Authoring basis	Active source, common Flyway migrations, recent feature and validation records
Application	MyAaptha; repository directory CircleNet
Status	Engineering baseline; organization approval and document owner to be assigned
Source counts	39 Java controller files; 229 extracted HTTP mappings; 28 web pages; 67 migration-declared tables
Distribution	Internal; contains structural information, no real customer records or credentials
Maintenance	Regenerate inventories after schema/route changes; review narrative at each release

Evidence hierarchy: executable source and migration files take precedence over historical manuals. Dated test reports are historical evidence, not a fresh test run. Planned infrastructure and performance targets are explicitly distinguished from observed behavior. Example names and records are fictitious.

Active scope: `backend/spring-boot-app`, `frontend/nextjs-app`, `mobile/flutter-app`, `ai/agent-service`, `infrastructure`. Older top-level MySQL starter implementations are reference material. The previous user/admin manuals contain aspirational items; do not infer social sign-in, offline mode, biometrics, unlimited circles or branch ACL from those manuals alone.

02 Architecture and technical boundaries



Platform architecture

Component	Responsibility	Source
Next.js 15 / React 19	Routes, accessible information controls, responsive cards, cookie-session client	`frontend/nextjs-app`
Flutter	Native screens, typed HTTP integration, lifecycle refresh, device capabilities	`mobile/flutter-app`
Java 21 / Spring Boot 3.5	Authentication, domain rules, owner checks, persistence, media and integrations	`backend/spring-boot-app`
PostgreSQL / Flyway	Relational persistence and versioned schema evolution	backend resources `db/migration/common`
Python / FastAPI	Candidate ranking, duplicate detection, insights, enrichment, contact organization	`ai/agent-service/src/ai_service`
Deployment tooling	Local lifecycle, container builds, Compose/Kubernetes templates, release gates	`infrastructure`

The backend is a modular application, not a separately deployed microservice per business domain. Controllers expose contracts; services apply business rules; repositories/JDBC persist state. The Python helper is a separate process. Internal AI helper endpoints must be restricted at the network boundary: its route definitions do not establish the browser's account authorization model.

Trust boundaries: client to API; API to database; API to object storage/scanner; API to external providers; operator to secrets and deployment tooling. Authorize access before fetching protected resources. Profile image GET routes are explicitly public in the security configuration; this exception must not be generalized to chat attachments or private documents.

03 Product capabilities and synchronization

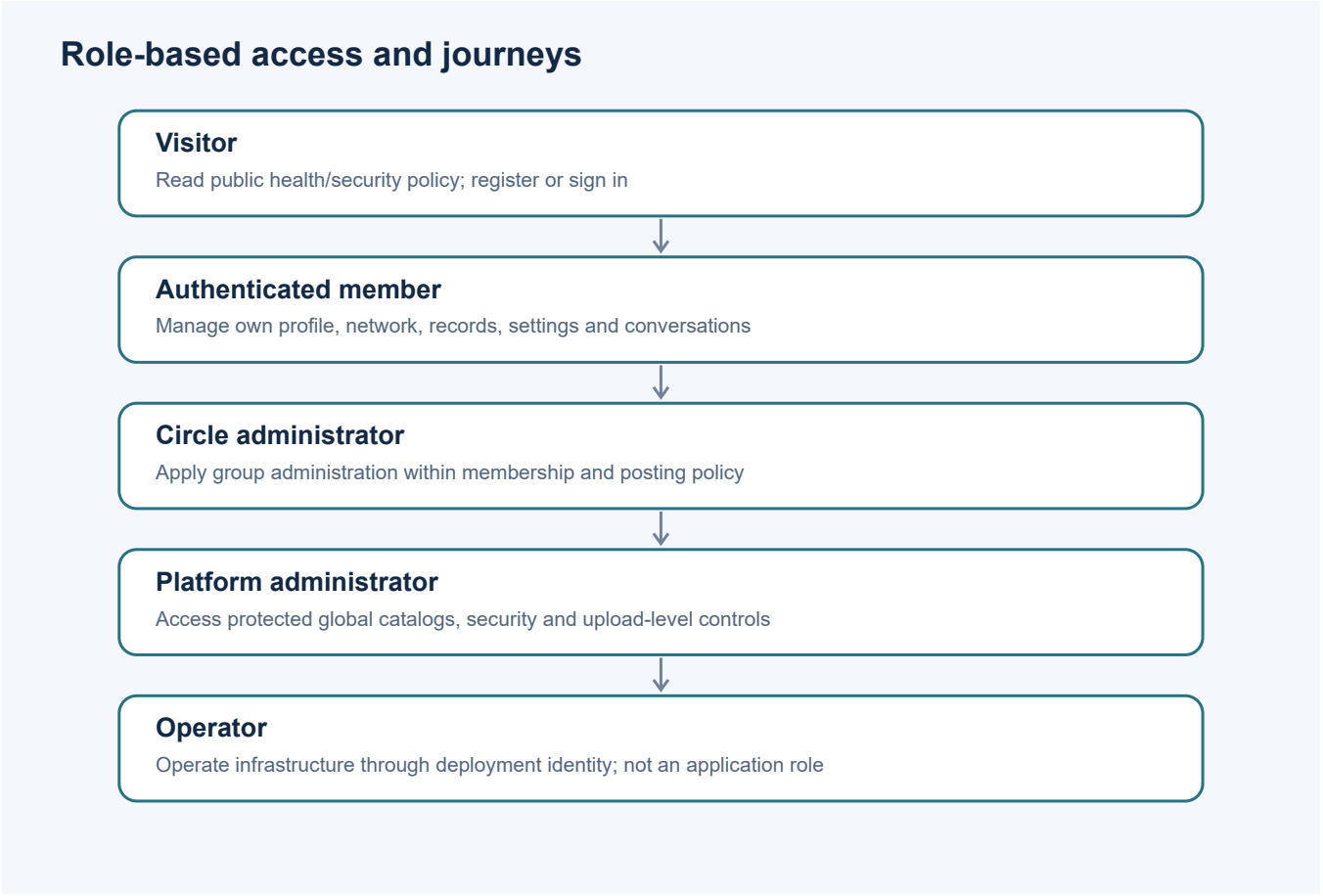
Capability	User value	Contract / source	Delivery qualification
Profile	Personal, address, education, employment and social details	`/api/profile`	Saved overview; section editing; latest-record merge
Relationship tree	Organize family and other relationship links	`/api/network`, people APIs	Owner-scoped data and relationship anchors
Address book	Import and review contacts	`/api/address-book`	Provider OAuth requires credentials; device permissions apply
Timeline	View milestones newest first; add or edit an event	`/api/timeline/events`	Shared saved-event API; client composition of derived milestones
Circles and messages	Group and direct conversation	network and chat APIs	Membership checks; read, edit and deletion rules
Diary / social	Record and share memories under visibility controls	`/api/social`	Distinct from timeline overrides
Finance	Review transactions and monthly summaries	`/api/finance`	UTC month handling across clients
Health	Store reports and request optional analysis	`/api/health`	Manual entry is independent; external analysis requires deployment enablement and per-report consent
Trust center	Follow role models; designate trusted people; emergency access	`/api/trust`	Two independent approvals and time limits
Settings	Discovery, notifications, workspace and account preferences	`/api/settings`	Storage limits come from server policy
AI workspace	Ask for assistance and review proposals	`/api/ai`	Consent/preferences and approval boundaries
Administration	Security, storage levels, moderation and global catalogs	admin/security and domain controllers	Server role and service checks

Synchronization contract: PostgreSQL is authoritative for persisted account data. Web uses its browser-session transport; Flutter uses bearer tokens and refresh. Refresh on relevant lifecycle transitions and after successful mutations. Equivalent behavior does not require identical layouts. The source-derived endpoint appendix identifies both transports' shared resources.

There is no evidence here of continuous real-time replication or complete offline conflict resolution. Polling/fetch, focus/resume and explicit refresh can leave a short stale-view interval. Profile sections merge against the latest fetched record to preserve other sections; concurrent writes within the same section are not guaranteed conflict-free. A full feature parity certification requires a release-by-release screen/contract matrix.

Native profile photo/gallery controls now use the shared profile API; the web timeline's parent-details interaction remains a client-specific behavior. Mobile-width browser screenshots test responsive web, not a native installed device. Native HTTP regression evidence validates the API client; APK compilation does not prove store readiness or every device behavior.

04 Roles, authorization and user journeys



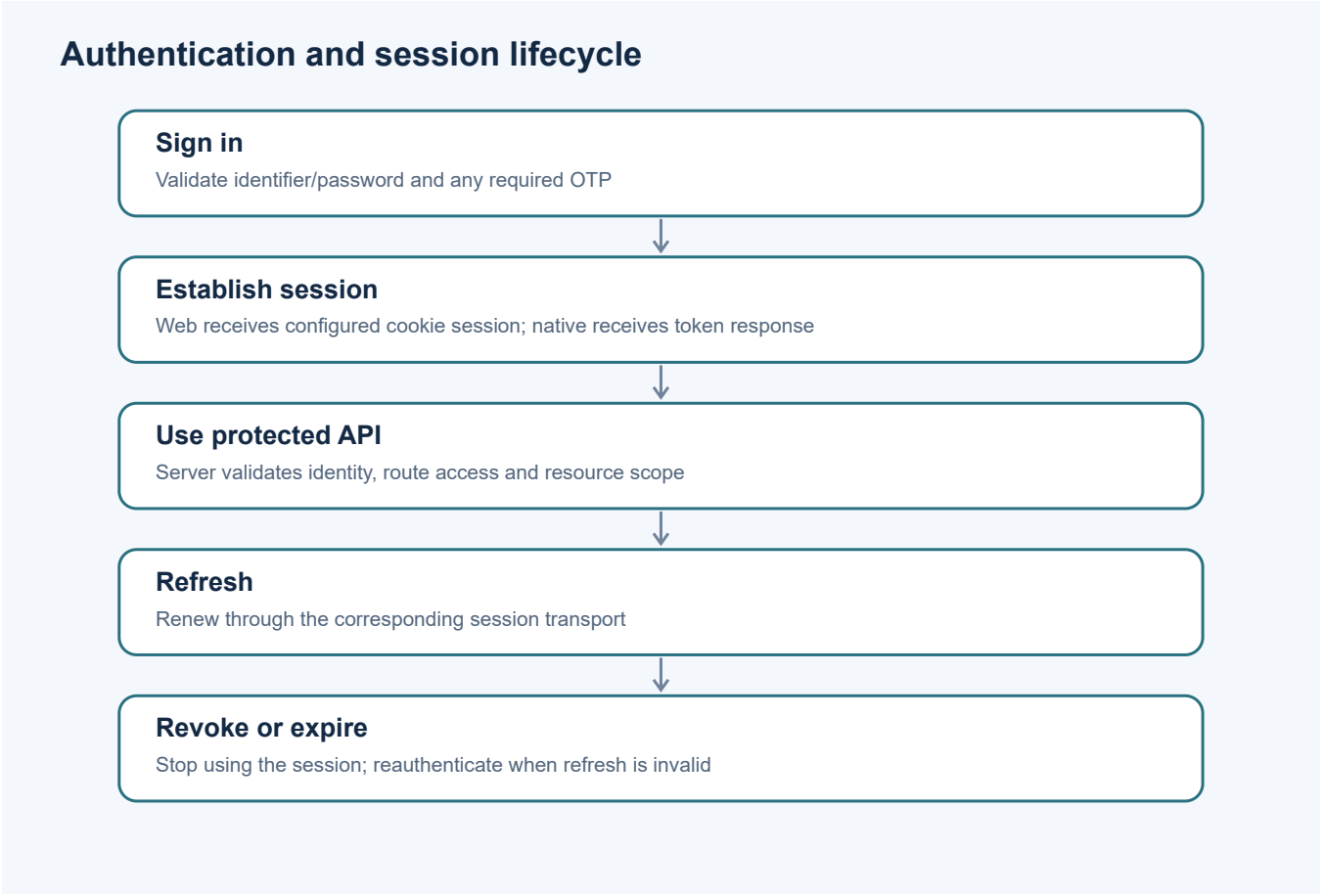
Role-based access and journeys

Application admin authority, circle admin membership and an infrastructure operator's permissions are different scopes. Do not assume an organizational tenant role hierarchy from the earlier generic admin manual. Consult `platform/config/SecurityConfig.java` and each service for actual checks.

Actor	Main flow	Guard / exception
New member	Register → authenticate → complete profile → add relationships → configure discovery	Validation, credential policy and session checks
Returning member	Sign in → dashboard → chosen personal workspace → save → refresh	Expired session refresh or sign in again
Circle member	Open permitted circle → fetch conversation → post if allowed	Admins-only posting can deny ordinary members
Circle admin	Review group → manage permitted settings → moderate eligible message	Delete-for-all remains time/read constrained
Star Member	Receive emergency request → approve/reject independently	One approval does not grant access
Platform admin	Authenticate → pass configured security checks → inspect admin workspace	Generic mutation routes may require admin; ownership still applies
Operations engineer	Read release gates → configure environment → validate → deploy → monitor	Deployment identity, backup and approval are external controls

Authorization sequence: reject invalid/expired authentication; verify route-level role; verify resource ownership or membership; validate action eligibility; execute within the required transaction; return only permitted fields. Hiding a button is presentation, not authorization.

05 Authentication, privacy and security



Authentication and session lifecycle

The API uses BCrypt password hashing and JWT-based stateless request authentication with persisted session/refresh controls. Native `/api/auth` and browser `/api/web-auth` have different transport semantics; do not move browser tokens into persistent JavaScript storage as a documentation shortcut. The login DTO accepts `identifier`, compatibility `email`, `password` and optional six-digit `otpCode`.

Spring's standard CSRF feature is disabled for the JWT application; the browser-cookie boundary uses its own origin/header checks. Review that filter and production origin configuration together. Security headers include CSP restrictions, referrer controls and HSTS. CORS and TLS need deployment-specific configuration.

Public registration, sign-in/refresh/logout/revoke, policy, callback and health routes are deliberate exceptions. Profile media and notification unsubscribe have specific public route exceptions. Most APIs require authentication; global administration routes require admin authority; personalized services additionally enforce owner/membership restrictions.

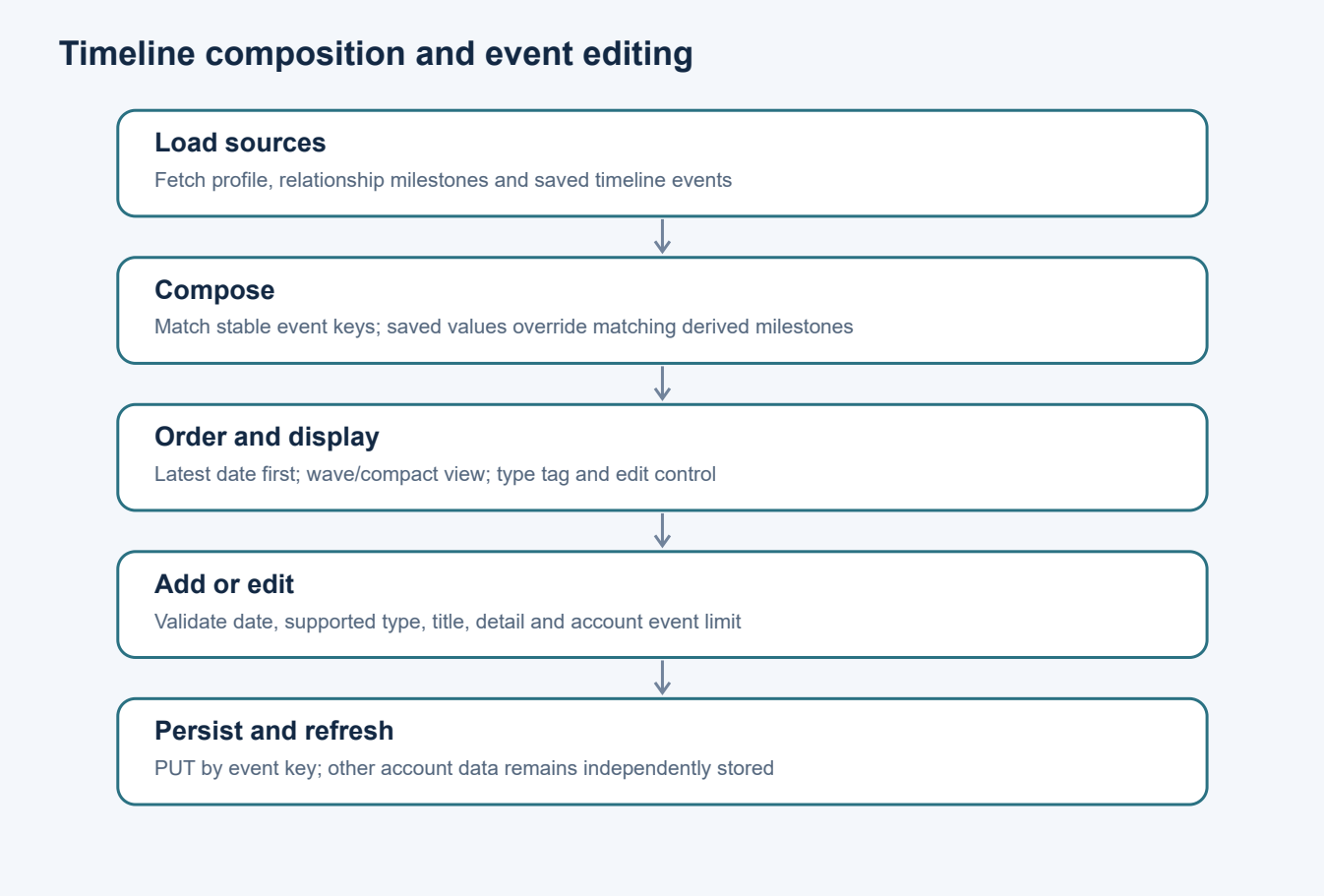
Data classification: credentials and session secrets are restricted; contact, health, finance and private messages are confidential; public profile media is publicly retrievable by its route policy; operational metrics and schema metadata are internal. Never include actual account exports, tokens, provider secrets or signed URLs in this handbook.

Review checklist for a release: cross-account denial, group membership changes, token revocation, cookie-origin rejection, protected attachment access, upload scanning failures, abuse-report authorization, sensitive-field minimization and audit retention. These are verification obligations, not a claim of universal certification.

06 Profile, settings and timeline workflows

Profile: display saved values by default. An Edit action opens the corresponding section form; Save updates that section against a newly fetched profile; Cancel returns to the saved overview. Invalid image URLs fall back to initials. Photo upload validates its own smaller limit; general storage limits must not be substituted for the photo policy.

Settings: fetch account preferences and upload policy; compute storage percentage from used bytes and allowance; show usage in a circular indicator with capacity and per-file limit below. Clamp the displayed ring to a valid range while reporting actual usage accurately. Administrators can assign configured upload levels; clients must not hardcode entitlements.

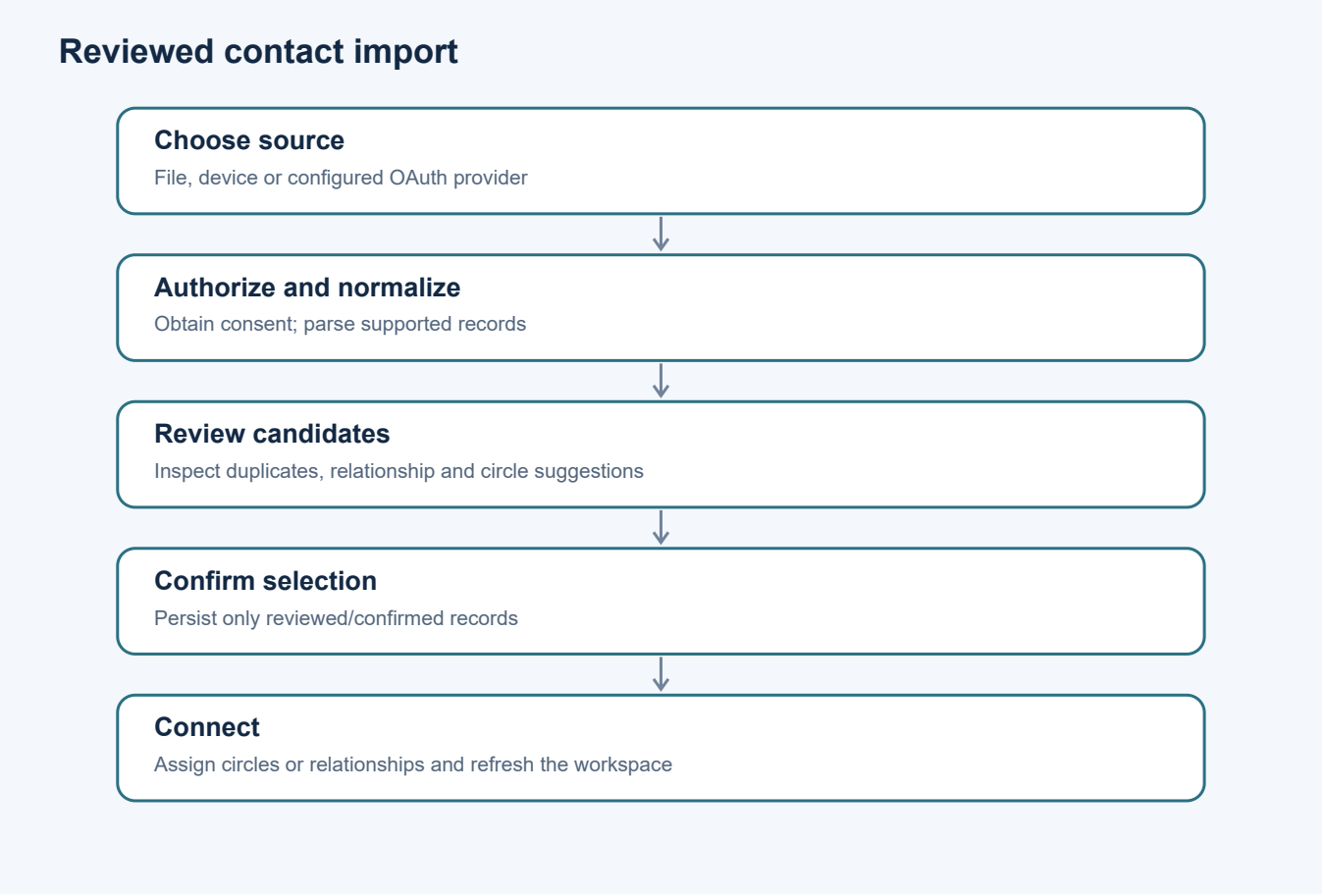


Timeline composition and event editing

Rule	Implemented behavior
Types	Birth, Family, Remembrance, Education, Marriage, Employment, Other
Colors	Blue birth/family; red remembrance; purple education; pink marriage; amber employment
Stable keys	`profile-birth`, `profile-education`, relation-based birth/death/marriage keys
Saved event scope	Composite `(user_id, event_key)` identity
Key validation	Letters, numbers, underscore and hyphen; 1–100 characters
Date	Valid ISO date; storage is VARCHAR(10), validated by LocalDate
Content	Nonblank title ≤200; detail ≤2,000 characters
New event ceiling	500 saved events per user; existing event updates remain possible
Delete	Owner-scoped DELETE removes a saved event or override; source milestones remain intact

An education year is displayed using a December 31 date convention; this is not evidence of the actual graduation day. Editing a derived milestone creates/updates a timeline override and does not alter the source profile or relationship. Source records can therefore intentionally differ from a customized timeline label.

07 Relationships, contacts and circle workflows



Reviewed contact import

Member workflow: add a managed person or import a contact → review identity and contact fields → choose relationship and anchor → save → inspect the tree → correct ambiguous links. A managed person record is not automatically a registered login account. Keep `users`, personal people records and relationship edges conceptually distinct.

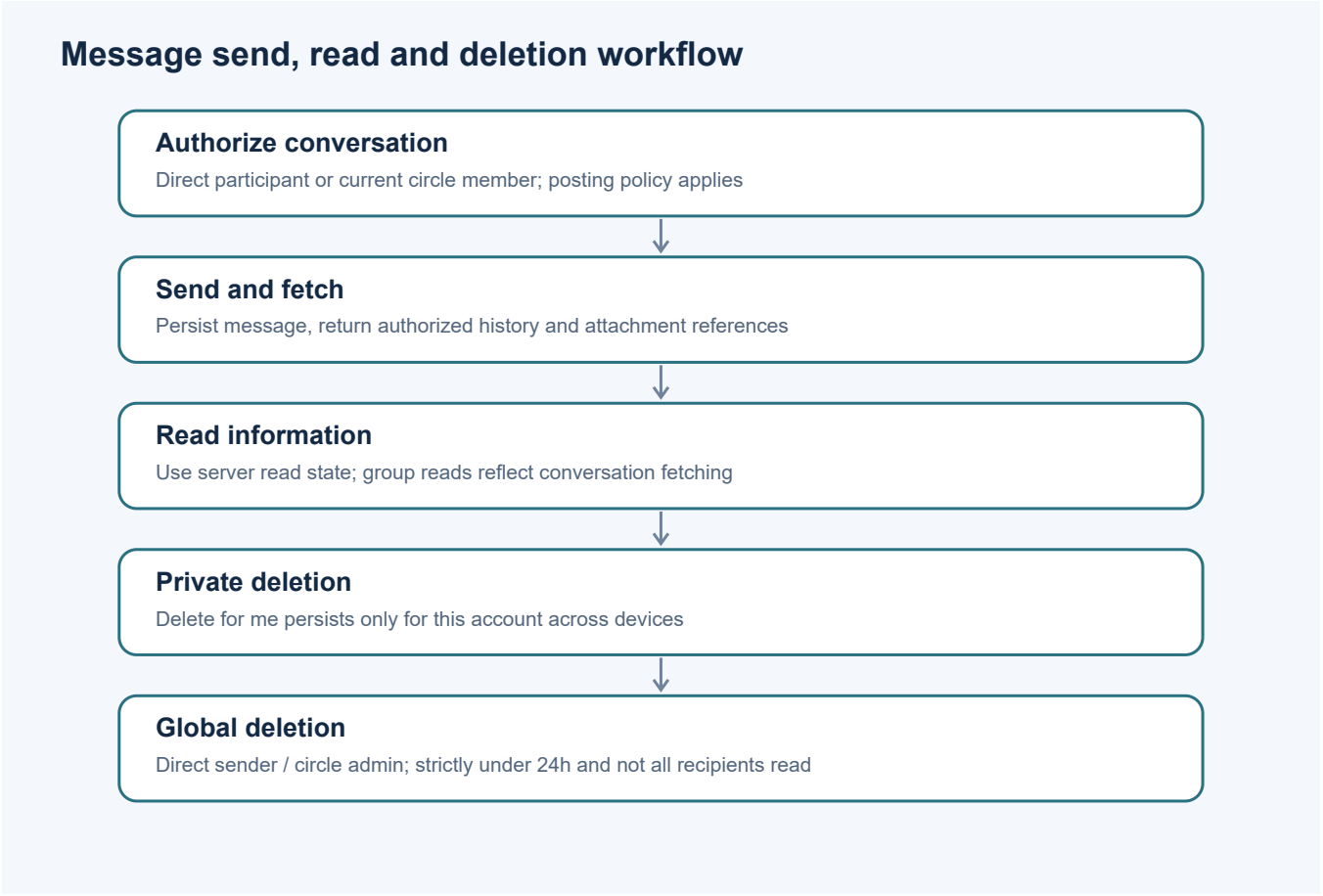
Contact import: choose a supported file/device/provider → obtain permission or OAuth consent → parse and normalize → inspect duplicates and suggestions → confirm selected entries → persist → optionally assign suggested circles. AI organization produces reviewable suggestions; it is not permission to silently merge identities. Google/Microsoft OAuth credentials, redirect URLs and outbound connectivity are configuration prerequisites.

Relationship example: Asha adds Ravi as a parent, anchored to herself, then adds Meera as a sibling under the correct shared parent context. A connection label depends on direction and anchor; reversing an edge can change its meaning. Review dates independently before deriving birth or remembrance milestones.

Circle workflow: create or select a permitted group → configure membership/admins/posting policy → fetch conversation → post if permitted → inspect read/message information → moderate according to server rules. Membership is the security boundary; a UI link does not grant access.

Data quality controls: avoid treating the same email/phone as conclusive identity without human review; preserve original import context; inspect duplicate candidates; confirm anchors; keep private contact details out of people-discovery cards. Role-model discovery showing generated-looking usernames may reflect existing test/generated account records; labels alone do not prove an account is a real person or endorsed role model.

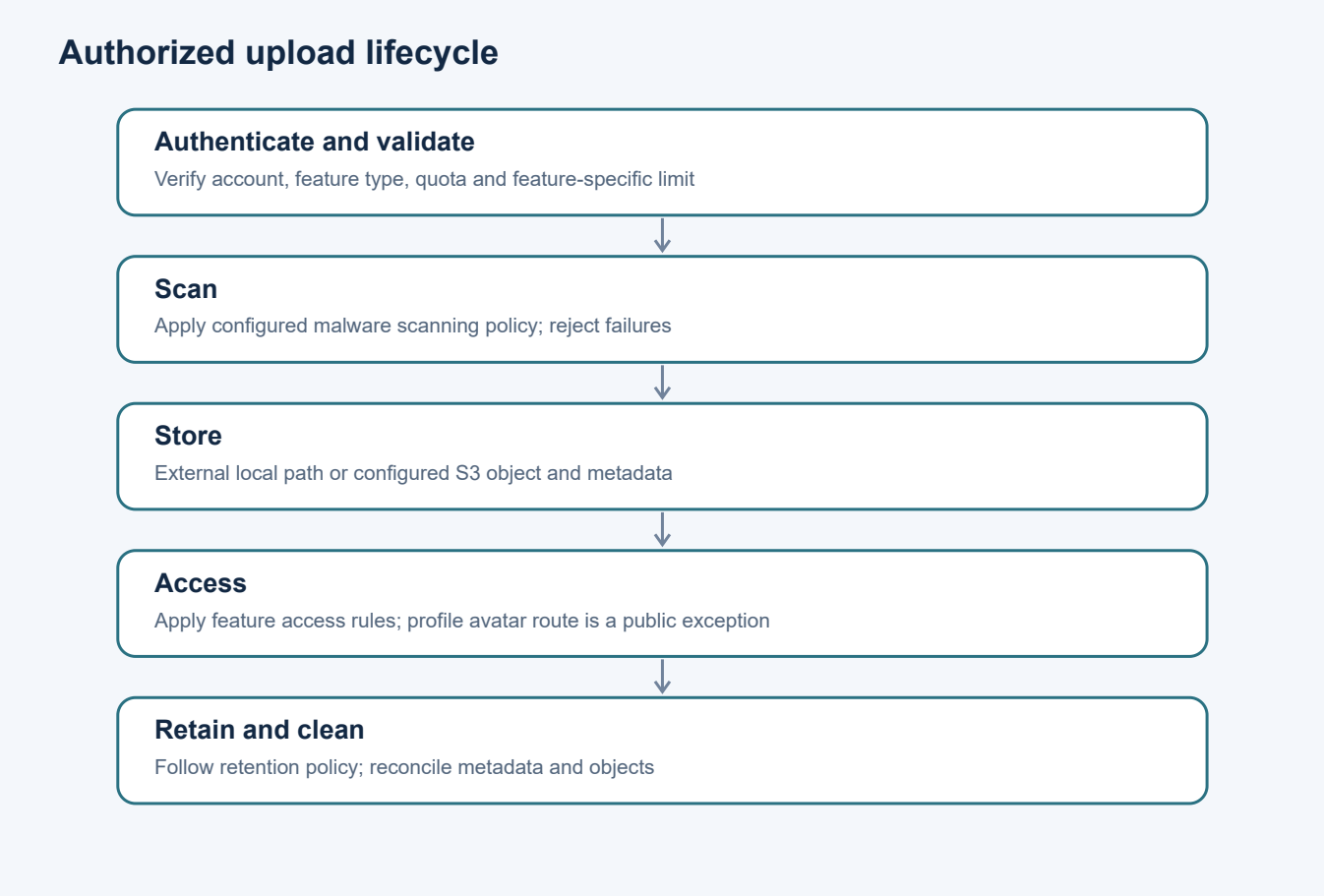
08 Messaging, media and notifications



Message send, read and deletion workflow

Delete-for-me is private and persisted across devices. In direct conversations only the sender can delete for all. In circles only current circle admins can delete for all, including another member's post; ordinary members cannot globally delete their own group post. Eligibility expires when all recipients have read or 24 hours elapse. Hidden read receipts do not erase stored direct read state. Group reads currently reflect conversation fetch, not per-message viewport acknowledgement.

Hidden markers allow cursor history merging to avoid restoring privately deleted content. Search, saved messages, unread counts and conversation previews exclude private deletion. Attachment access and forwarding reject hidden direct-message sources. Locking/transactions enforce eligibility against concurrent reads and edits.

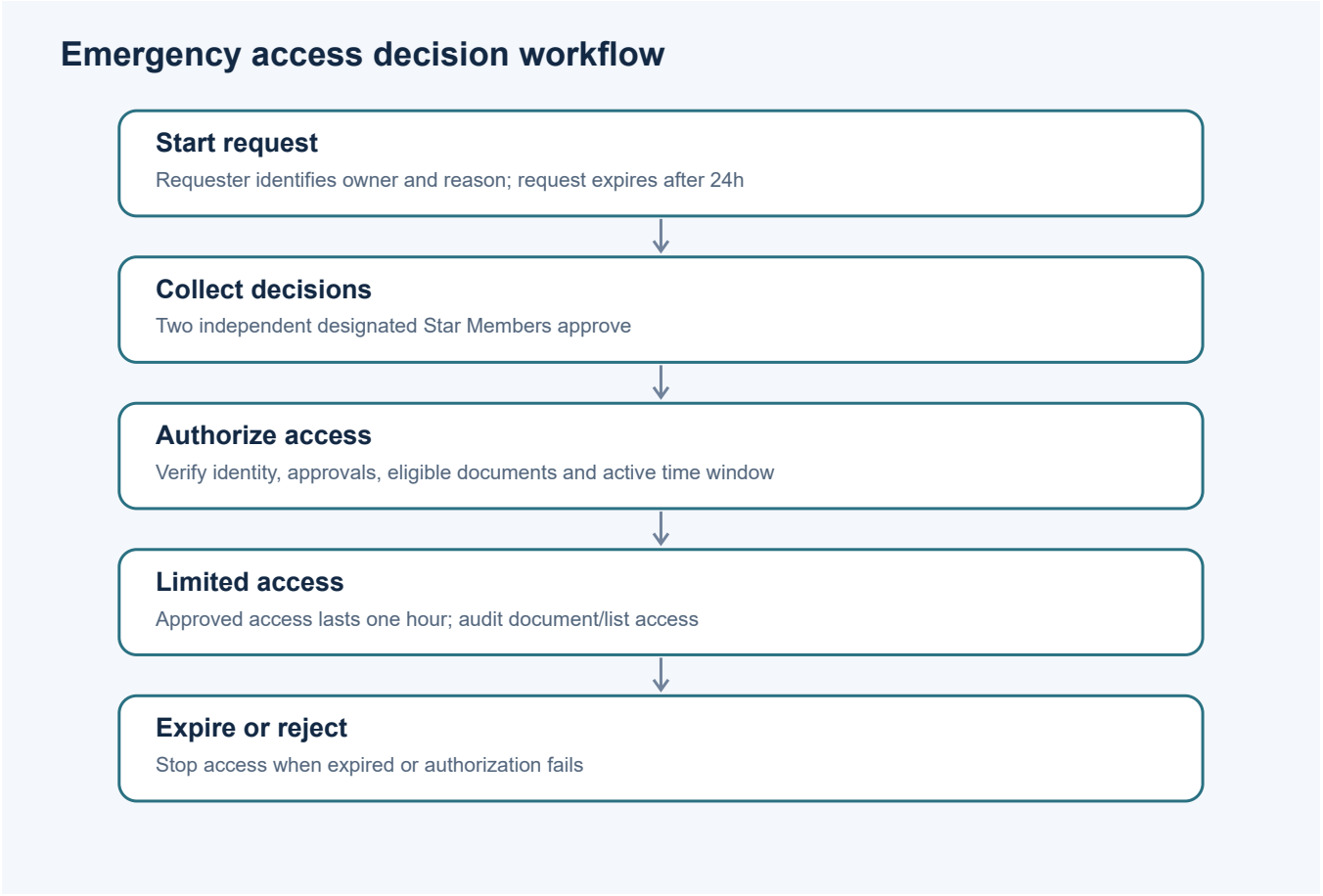


Authorized upload lifecycle

Upload lifecycle: authenticate → check feature-specific size, quota and allowed type → validate content → scan under configured policy → store outside the application package or in configured object storage → record metadata → authorize later access → retention cleanup removes eligible media. General multipart configuration allows 25 MB per file; profile policy is smaller (5 MB). Request/container/gateway limits are separate controls and must be checked together. Scanner and storage failures must not be presented as successful persistence.

Notification lifecycle: an application event produces a recipient notification; preferences and delivery configuration govern channels; the dispatcher attempts configured delivery and records result/retry handling. The default dispatcher interval is five seconds. A stored in-app notification does not prove external SMS/email/push delivery. Device registration, unsubscribe links and provider configuration require separate validation.

09 Trust, health, finance and AI workflows



Emergency access decision workflow

Two distinct designated Star Members must approve emergency access. Unrelated users and a single approval must remain denied. List views, decisions and document opens produce emergency audit events. Multi-channel owner alerting is an operational requirement requiring configured providers; do not infer it from the presence of the emergency API.

Health use case: member uploads a report → API validates access and file policy → store report → member supplies per-report processing consent and requests analysis only if enabled/configured → show interpretation with provenance and limitations → member reviews and can remove the report through authorized controls. Analysis is assistance, not a clinical diagnosis or an established regulatory approval.

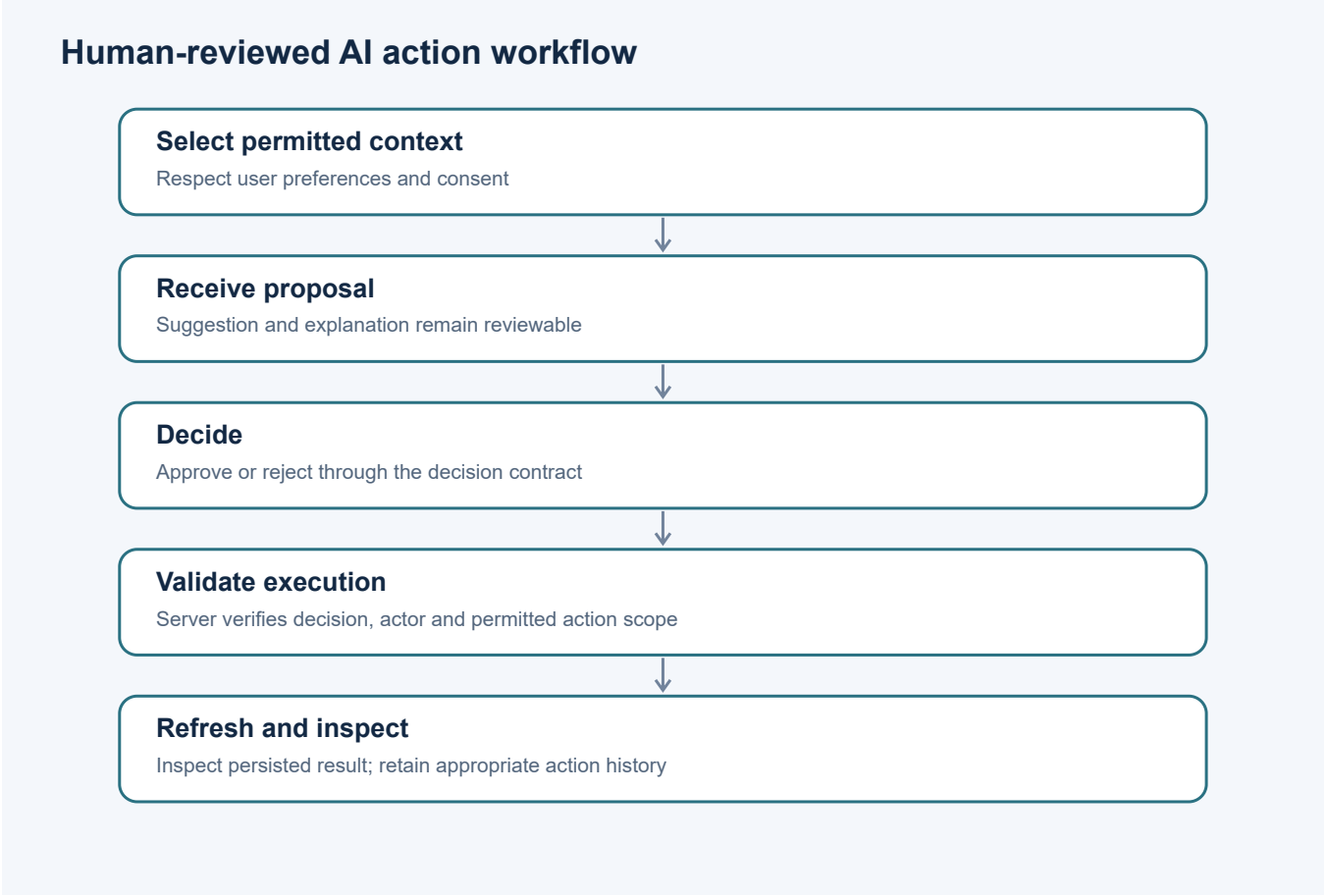
Manual health entry: web/native support multiple named measurements and units, optional reference ranges, collection date, notes and an optional owned diary link. API validation rejects missing values, future dates, non-finite values and inverted reference bounds. Stored limits are shared: report/lab 200, metric 120, unit 40 and notes 1,000 characters. Failed client saves preserve drafts; busy controls prevent concurrent clicks. Unknown network outcomes do not have a server idempotency guarantee. Optional external analysis defaults off and uses a separate consent path. See current health-entry contract.

Finance use case: import or enter supported transaction data → inspect parsed values → persist account-owned transactions → calculate monthly summaries in the shared UTC convention → correct incorrect classification → refresh browser/native summaries. Example: a fictitious expense of 1,250 on 2026-10-01 belongs to the October UTC reporting period; currency and parsing assumptions must be explicit in import validation.

AI use case: fetch preferences/consent → submit permitted context → receive explanation or proposed action → inspect proposed change → approve or reject through the appropriate approval API → execute only after the server validates decision, actor and action scope → inspect resulting data. Not every response requires execution; suggestion endpoints do not imply autonomous writes.

The Python helper's ranker uses normalized fields and similarity; duplicate detection compares phone/email and similar names; family insights and enrichment require explicit consent in their request contracts. Contact organization uses rules/aliases. These deterministic endpoints do not establish a

trained proprietary model. External LLM-backed help/health functionality is a separate configured backend integration. Approvals, execution and history have dedicated Java controllers.



Human-reviewed AI action workflow

10 Use-case catalog and acceptance scenarios

ID	Actor / trigger	Main result	Alternate or denial path	Acceptance evidence to capture
UC-01	Visitor registers	Account eligible for sign-in	Invalid/duplicate data rejected	Registration validation and credential policy
UC-02	Member edits education	Saved overview reflects section change	Cancel preserves saved values	Other profile sections remain unchanged
UC-03	Member adds timeline event	Event visible in latest-first order	Invalid date/type/title rejected	Reload and native fetch show saved value
UC-04	Member overrides birth milestone	Timeline label/date overridden	Source profile stays independently stored	Matching stable key and source comparison
UC-05	Member imports contacts	Reviewed entries persisted	Missing consent/provider error/duplicate warning	Confirmed selection, no silent merge
UC-06	Member links a relative	Tree and milestone context update	Invalid anchor or inaccessible person denied	Direction and owner isolation
UC-07	Circle member sends message	Authorized group sees message	Admin-only posting denies ordinary member	Membership and posting-policy tests
UC-08	Member deletes for me	Sender's history hides entry	Other participant retains entry	Cross-device reload, search and attachment access
UC-09	Sender/admin deletes for all	Eligible message disappears/redacts	Read/time/role constraint denies	Boundary at 24h, race with read acknowledgement
UC-10	Member uploads report	Stored authorized metadata/media	Quota/type/scanner failure denied	No unscanned successful upload
UC-11	Stars approve emergency	Limited access after two approvals	One approval/unrelated actor/expiry denied	Audit trail and time-window checks
UC-12	Member approves AI proposal	Validated approved change executes	Rejection or changed scope blocks	Decision/actor binding and resulting state
UC-13	Admin changes upload level	API returns updated entitlement	Non-admin denied	Server and both-client policy refresh
UC-14	Operator restores backup	Isolated restored environment validated	Failed checks prevent cutover	Actual measured RPO/RTO and integrity report

Example end-to-end scenario: Asha signs in, saves education, adds a graduation event, imports reviewed contacts, opens a family circle and posts a reunion message. Her Flutter client refreshes the saved event and the same account preferences. Ravi, using another account, must not fetch Asha's private event list or health report by changing an identifier. Record API responses, account isolation, refresh behavior and UI screenshots separately.

Boundary scenarios: expired refresh token; two edits to different profile sections; same-section concurrent edits; invalid leap date; 501st new timeline row; edited existing row at the limit; removed circle member requesting old attachments; all recipients read immediately before deletion; failed malware scan; exhausted storage; provider outage; emergency expiry; missing external AI consent.

11 Data architecture and relationship model

The physical system uses PostgreSQL migrations. `users` represent login identities; personal/managed people records and relationship edges represent the member's network; circles and membership tables represent communication scope. Other domains attach account-owned settings, sessions, media, records, notifications, AI decisions and trust events.

Physical versus logical relationships: the ER files below are extracted from explicit migration `REFERENCES` clauses. Ownership columns without a declared FK are business relationships and must not be shown as database-enforced constraints. Optional references may be zero-or-one; the overview deliberately uses conservative optional cardinality. Uniqueness and nullability details are in the migration inventory.

The schema appendix is a source-derived DDL reference, not a live `pg_catalog` export. It tracks CREATE TABLE, ADD/DROP COLUMN and retains ALTER/constraint statements for review. Complex conditional SQL, renamed structures or altered definitions must be reconciled against PostgreSQL metadata before treating it as a certified deployed schema. Runtime database access was not required to prepare this package.

Retention and integrity: follow documented media retention and privacy workflows; do not assume row deletion removes every external object. Timeline ownership FK has ON DELETE CASCADE. Session, audit and emergency records have independent purposes and retention requirements. Validate backups jointly with object storage and migration version.

12 API contracts and integration examples

The source-derived appendix enumerates method/path/controller for active Java mappings. It is a route catalog, not a complete OpenAPI contract: body validation, query parameters, media types and error envelopes remain defined in controller DTOs/services. AI helper routes are separately cataloged. Check `SecurityConfig.java` in declaration order before assigning a permission label to a path.

Native sign-in example (fictitious credentials):

```
POST /api/auth/login
Content-Type: application/json

{"identifier":"asha@example.invalid","password":"<user-supplied-password>"}
```

Use the response's access token only through the authorized bearer transport. Browser login uses its cookie-session route and required origin/header policy. Do not assume the two login responses are interchangeable.

Create/update a timeline event:

```
PUT /api/timeline/events/custom-graduation-2026
Authorization: Bearer <access-token>
Content-Type: application/json

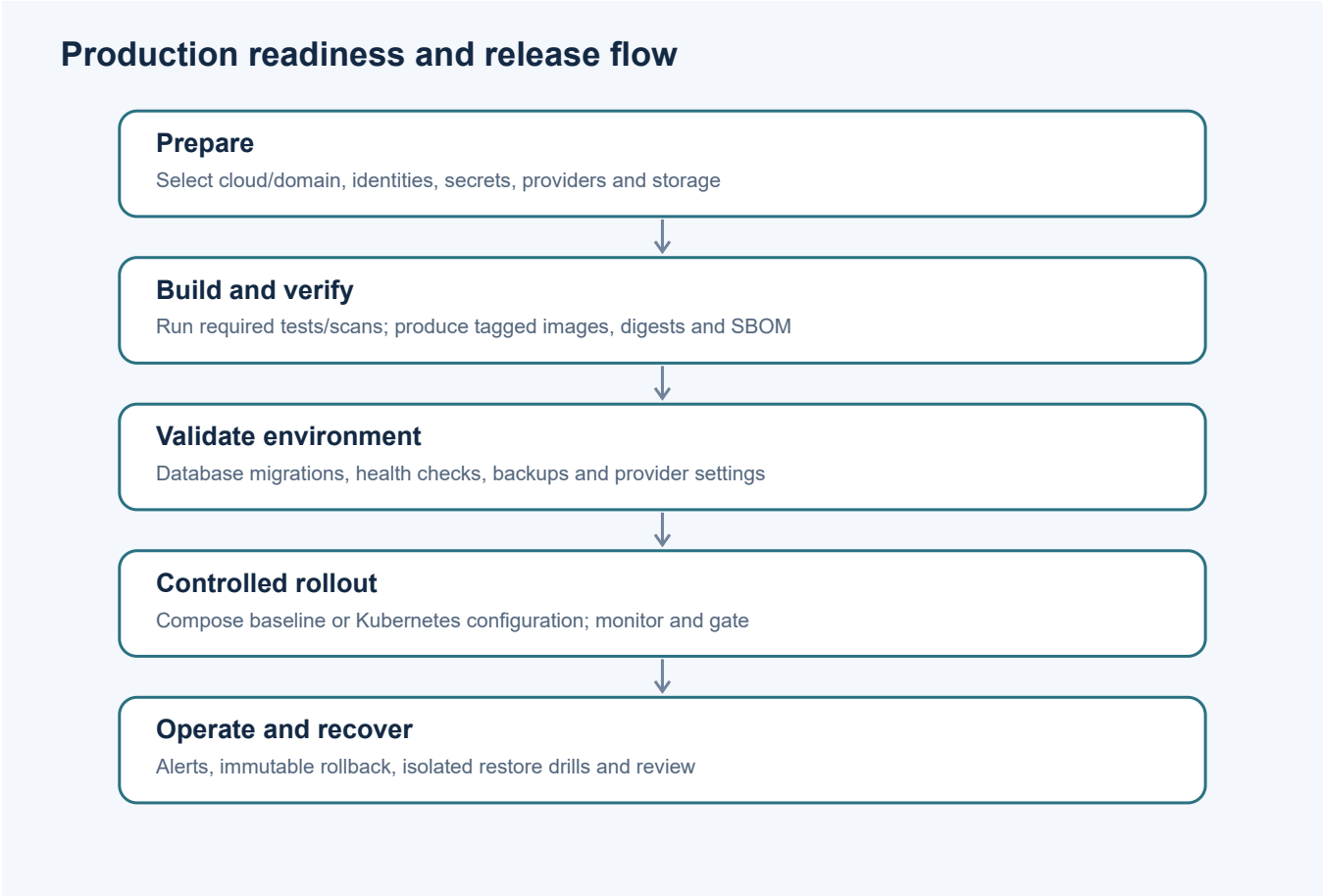
{"eventKey":"custom-graduation-2026","date":"2026-06-
15","kind":"Education","title":"Graduation","detail":"Completed a postgraduate course"}
```

The path key determines the saved event identity. GET `/api/timeline/events` returns the signed-in user's saved events ordered by date descending and key; it does not return the entire client-composed set of source milestones. Unsupported kind, invalid date or length violations return a bad request. The controller updates an existing owned key or inserts a new row. DELETE `/api/timeline/events/{key}` returns 204 and removes only the signed-in account's saved row or override; removing an override restores its source-derived display without changing profile/relationship records.

Error handling: 400 denotes validated bad input in the timeline contract; 401 authentication failures; 403 authorization denial where enforced; other controllers may return different statuses for missing or invalid resources. Clients should handle expiry, validation and transient integration failure separately. Do not assume an undocumented universal error envelope or blanket retry/idempotency guarantee for all POSTs.

Contract evolution: prefer additive changes, keep compatibility fields deliberately, validate strict JSON behavior and release clients together when contracts change. Test both cookie and bearer paths, cross-account denial and reload persistence. Timeline PUT is a stable-key update; arbitrary create/send operations may create duplicates if retried without an idempotency contract.

13 Deployment, configuration and operations



Production readiness and release flow

Local preview: run `powershell -ExecutionPolicy Bypass -File infrastructure/start-local.ps1`; `-SkipBuild` uses existing built artifacts. Stop through `infrastructure/stop-local.ps1`; the local PostgreSQL volume is retained. Canonical web preview uses port 3000; API and helper settings come from the local startup configuration. Do not start a competing Next.js dev build over a standalone preview's `.next` output.

Production baseline: Compose templates support a single-host TLS/Caddy deployment; Kubernetes templates provide replicas, probes, resources and autoscaling scaffolding. Managed PostgreSQL, Redis, object storage, secret manager and ingress are deployment dependencies, not proof that resources have been provisioned. Do not claim production readiness from local startup.

Configuration group	Required review
Database	URL, credentials, profile, migration locations, connection limits, backup/PITR
Authentication	Random managed JWT/session secrets, lifetimes, origin/cookie policy, admin security
Storage	Local/S3 provider, external path/bucket, IAM, quota, scanner, cleanup schedule
Contacts	OAuth credentials and exact redirect URLs for enabled providers
AI	Helper URL, provider credentials/model policy, consent and data egress
Notifications	Channel/provider credentials, webhook settings, device setup and retries
Infrastructure	Domain/TLS, registry digest, replicas, resources, observability and rollback
Mobile release	Signing identity, store metadata, permission declarations and device validation

Never promote local fallback credentials or secrets into production. Example configuration belongs in templates with placeholders; operational credentials belong in the approved secret mechanism.

The existing operations runbook states **targets**, including 99.9% monthly availability, RPO 15 minutes and RTO four hours. These are not measured guarantees. Baseline alert proposals include sustained 5xx rate, latency, database pool pressure and notification failures. Instrument the deployed system and agree organization-specific objectives before treating them as service commitments.

Recovery workflow: identify incident → preserve audit/log evidence → isolate affected component → revoke/rotate compromised access if needed → restore PostgreSQL into an isolated environment → validate migration version, constraints and representative domain access → reconcile media → measure recovery time and data loss → approve cutover → document follow-up. An image rollback alone cannot undo every schema/data change.

14 Quality assurance, evidence and release acceptance

Layer	Required checks	Existing evidence basis
API	Domain tests, authorization, input validation, persistence and migration compatibility	Recent sync/timeline/backend reports
Web	Production build, unit/e2e/integration, keyboard and responsive checks	October 8 app-design record
Flutter	Analyzer, unit/widget tests, HTTP contract integration and debug build	October 8 native client record
Providers	OAuth callback, delivery, AI consent and outage behavior	Requires configured deployment
Security	Dependency/image scans, secrets, isolation, malware handling	Release tools and security docs
Operations	Health/readiness, realistic capacity, backup restoration, rollback	Requires deployed environment and measured drill

Recent recorded evidence includes a passing web production build and 12 production E2E checks, Flutter analysis with no issues, 34 passing mobile tests with one opt-in live test skipped in the ordinary suite, and a separately passing native HTTP live check. These are **October 8 historical results**, not tests rerun during documentation generation. The browser route review covered 28 routes at desktop/mobile sizes; native device behavior is a separate obligation.

Earlier API and synchronization reports contain different test counts from different dates/scopes. Do not add them together as a current suite size. Consult linked evidence and rerun the release's required checks from its exact commit before sign-off.

Release acceptance: identify release artifact and migration set; validate both session transports; verify cross-account isolation; cover file/quota limits; verify derived and saved timeline composition; validate message deletion races; test configured providers; record accessibility and device checks; confirm secrets, backup restoration and rollout/rollback gates. Open issues require an accountable owner and a recorded acceptance or remediation decision.

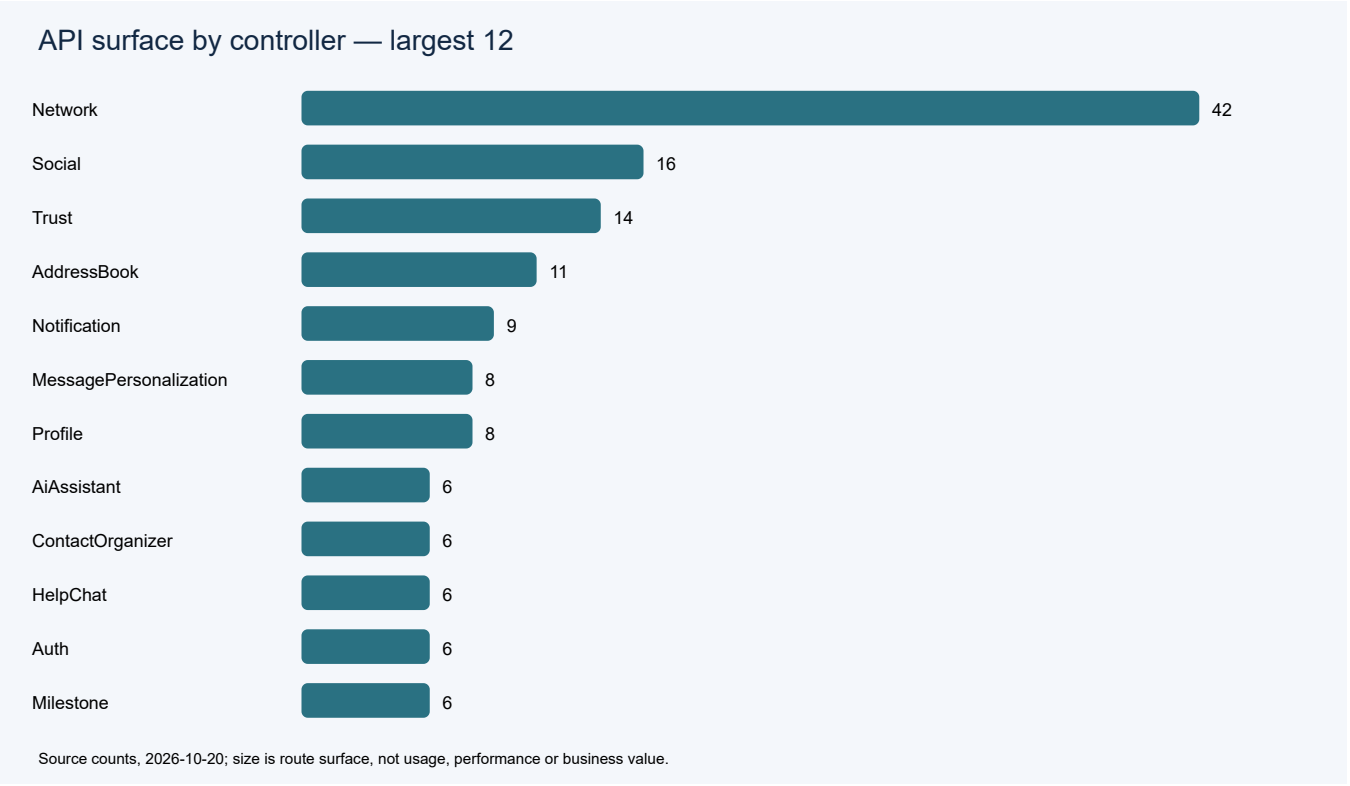
15 Known limitations, decisions and maintenance

Item	Impact / decision
Historical aspirational manuals	New handbook is the implementation-grounded entry point; retain old records with context
Configuration-dependent providers	Do not advertise delivered notifications, OAuth import or external AI without configuration tests
Timeline saved overrides	Owner-scoped saved-row deletion/reset; source profile/relationships remain independent; education date convention
Same-section concurrent profile edits	Latest merge protects other sections; no universal conflict resolution guarantee
Client differences	Shared API does not establish complete feature-identical web/native experiences
Public profile image route	Treat avatar imagery according to actual public route policy
Physical DDL inventory	Review raw migrations and deployed catalog for complex ALTER/conditional statements
Capacity/SLA	Existing targets and historic local measurements do not establish production guarantees
Cloud/store handoff	Infrastructure provisioning, secrets, restore drills and mobile signing remain external release gates

Change process: update source and tests → add migration if required → regenerate this package → review route/data diffs → update diagrams and user scenarios → validate portable rendering and links → assign document version/approval → publish through the organization's approved channel.

Glossary: account = authenticated user identity; managed person = network record that may not have a login; relationship = directed/anchored connection; circle = communication/membership group; Star Member = designated trust actor; override = saved timeline value taking precedence over a matching derived key; upload level = configured storage entitlement; evidence = a dated recorded verification outcome; target = desired service behavior requiring measurement.

16 Source inventory, graphs and references



Source-derived API surface graph

The graph counts extracted method/path mappings, including transport and administration endpoints. It measures source surface only; it does not measure demand, throughput, quality or delivery completeness.

Web route inventory

- `/.`
- `/ai-activity`
- `/auth`
- `/circle-search`
- `/circles`
- `/contacts`
- `/dashboard`
- `/discover`
- `/feed`
- `/finance`
- `/following`
- `/health`
- `/help`
- `/messages`
- `/moderation`
- `/notifications`
- `/people/[id]`
- `/privacy`
- `/profile`
- `/recover-account`
- `/relationship-tree`

- `/reports`
- `/security-reports`
- `/session`
- `/settings`
- `/timeline`
- `/trust`
- `/verify-phone`

Appendices

- API catalog: Java method/path/source links and Python helper routes.
- Data dictionary: table declarations, added columns and ALTER history.
- ER diagrams: whole-system and domain-specific Mermaid relationships.
- Machine-readable inventory: repeatable route/schema metadata.
- Physical FK map: standalone vector reference.

Authoritative and supporting references

- Project README
- Recent application design and validation
- Synchronization audit
- Timeline API and clients
- Profile and storage behavior
- Contact import
- Message deletion policy
- Trust test strategy
- Deployment handoff
- Operations runbook and targets
- Security configuration

17 Current engineering changes and acceptance boundaries

This edition refreshes source-derived API/schema references to 20 October 2026 and supersedes the 10 October handbook narrative where behavior changed. Earlier dated verification is preserved as historical evidence.

Area	Current implementation and evidence
Moderation	Admin-only status-filtered cursor queue on web/native/API; old reports remain reachable. Reporter-visible notes and cancellation behavior are explicit. V69 indexes the queue. Workflow and role boundaries.
Contacts	Shared native granted/limited permission boundary, review before import and no read/upload after denied permission or leaving the pending dialog. Local plugin/API checks pass; physical device acceptance remains open. Permission matrix. OAuth now handles terminal failures, empty results, cancellation and consent revocation consistently across clients; completion contract and staging limits.
OAuth storage	Pending authorizations and review results now use the configured shared state provider, encrypted Redis payloads, hashed keys, ten-minute TTL and atomic owner/provider-validated consumption. Memory remains development only; real provider/staging acceptance remains open. Storage and rollout workflow.
Health	Multi-measurement manual entry, matching bounds, preserved failed drafts and disabled-by-default consent-gated analysis. Contract.
Release identity	Version 2 hashes the full standalone web bundle and API/debug APK, with a verification command. Local identities never establish production promotion approval. Artifact scope.
Operational authorizatio n	Administrator-only Actuator metadata and an optional narrowly scoped metrics credential. Packaged production-profile TLS checks verified isolated PostgreSQL/Redis readiness withdrawal, retained liveness/metrics and recovery. Real collectors, alerts and live rollback remain pending; see operational guide.
Android release	Release builds require a protected owner signing identity and reject debug signing. A manual candidate workflow pins the certificate and records signature/packaging review results; synthetic-key local verification does not establish store acceptance. Signing implementation.
Recovery	Unique complete dump publication, mandatory checksum verification and nonempty-target rejection. Actual local two-row PostgreSQL drill passed; managed PITR/media/RPO/RTO remain open. Recovery scope.
AI evaluation	Ten synthetic review cases and offline comparison; actual stronger-model quality acceptance remains pending and retries default off. Rubric and workflow.

The viewable launch board and dated module status distinguish fully closed local tasks from partial engineering and external acceptance. Current scope includes explicitly identified engineering additions; changed scope is disclosed rather than presented as closure of provider, staging or approval work. Saved browser edits remain separate from the latest source evidence.

The 18 October development report records separate backend, native, browser, build and operational checks. Do not sum different test scopes into one suite size. Physical Android/iOS acceptance, selected staging deployment, real delivery/OAuth, independent security review, policy approval, capacity and production promotion remain open. The latest recorded 5,000-user capacity run failed. No paid stronger-model call or production deployment was performed for this update. The 20 October acceptance audit records the exact requirement, proposed team and next action for every open gate. Offline rollback preparation and Android signing evidence are in the 20 October report. The 19 October continuation records the shared OAuth storage migration and its verification boundaries.

Appendix A — API catalog

API route inventory

Source snapshot: 20 October 2026. 229 extracted mappings from 39 Java controller files. No runtime/OpenAPI contract claim.

Method	Route	Controller
GET	`/api/ai/approvals`	AiActionApprovalController
PUT	`/api/ai/approvals/{approvalId}/decision`	AiActionApprovalController
GET	`/api/ai/executions`	AiActionExecutionController
POST	`/api/ai/search/rank`	AiAssistantController
POST	`/api/ai/duplicates`	AiAssistantController
POST	`/api/ai/family/insights`	AiAssistantController
POST	`/api/ai/profiles/enrichment`	AiAssistantController
POST	`/api/ai/contacts/organize`	AiAssistantController
GET	`/api/ai/activity`	AiAssistantController
GET	`/api/ai/preferences`	AiPreferenceController
PUT	`/api/ai/preferences`	AiPreferenceController
POST	`/api/contact-organizer/analyze`	ContactOrganizerController
POST	`/api/contact-organizer/accept`	ContactOrganizerController
POST	`/api/contact-organizer/oauth/start`	ContactOrganizerController
GET	`/api/contact-organizer/oauth/callback/{provider}`	ContactOrganizerController
GET	`/api/contact-organizertext/html`	ContactOrganizerController
GET	`/api/contact-organizer/oauth/results/{resultKey}`	ContactOrganizerController
GET	`/api/ai/help`	HelpChatController
POST	`/api/ai/help/chat`	HelpChatController
DELETE	`/api/ai/help/history`	HelpChatController
PUT	`/api/ai/help/admin/settings`	HelpChatController
POST	`/api/ai/help/admin/answers`	HelpChatController
DELETE	`/api/ai/help/admin/answers`	HelpChatController
GET	`/api/auth/health`	AuthController
GET	`/api/auth/me`	AuthController
POST	`/api/auth/login`	AuthController
POST	`/api/auth/refresh`	AuthController
POST	`/api/auth/logout`	AuthController

POST	`/api/auth/revoke`	AuthController
GET	`/api/auth/security-policy`	PhoneVerificationController
POST	`/api/auth/phone/start`	PhoneVerificationController
POST	`/api/auth/reset-password`	PhoneVerificationController
POST	`/api/web-auth/login`	WebSessionController
POST	`/api/web-auth/refresh`	WebSessionController
POST	`/api/web-auth/logout`	WebSessionController
POST	`/api/web-auth/revoke`	WebSessionController
GET	`/api/circles`	CircleController
GET	`/api/circles/{id}`	CircleController
POST	`/api/circles`	CircleController
PUT	`/api/circles/{id}`	CircleController
DELETE	`/api/circles/{id}`	CircleController
GET	`/api/address-book/providers`	AddressBookController
POST	`/api/address-book/oauth/start`	AddressBookController
GET	`/api/address-book/oauth/results/{key}`	AddressBookController
POST	`/api/address-book/import-reviewed`	AddressBookController
POST	`/api/address-book/import`	AddressBookController
GET	`/api/address-book`	AddressBookController
GET	`/api/address-book/circles`	AddressBookController
PUT	`/api/address-book/{id}/circles`	AddressBookController
DELETE	`/api/address-book/{id}`	AddressBookController
DELETE	`/api/address-book`	AddressBookController
POST	`/api/address-book/{id}/relation`	AddressBookController
GET	`/api/dashboard/summary`	DashboardSummaryController
GET	`/api/finance/summary`	FinanceController
POST	`/api/finance/transactions`	FinanceController
DELETE	`/api/finance/transactions/{id}`	FinanceController
GET	`/api/health/ai-status`	HealthController
GET	`/api/health/dashboard`	HealthController
POST	`/api/health/reports`	HealthController
POST	`/api/health/reports/analyze`	HealthController
DELETE	`/api/health/reports/{id}`	HealthController
POST	`/api/network/chats/groups`	ChatDiscoveryController
GET	`/api/network/chats/search`	ChatDiscoveryController

POST	`/api/network/messages/with/{other}/{message}/forward`	MessagePersonalizationController
PUT	`/api/network/messages/with/{other}/{message}/flags`	MessagePersonalizationController
GET	`/api/network/messages/with/{other}/saved`	MessagePersonalizationController
POST	`/api/network/messages/with/{other}/read`	MessagePersonalizationController
PUT	`/api/network/messages/with/{other}/pin`	MessagePersonalizationController
PUT	`/api/network/messages/with/{other}/settings`	MessagePersonalizationController
GET	`/api/network/messages/preferences`	MessagePersonalizationController
PUT	`/api/network/messages/preferences`	MessagePersonalizationController
GET	`/api/network/chats/circles`	UnifiedChatController
PUT	`/api/network/chats/circles/{circleId}/{messageId}/flags`	UnifiedChatController
POST	`/api/network/chats/forward`	UnifiedChatController
GET	`/api/milestones`	MilestoneController
GET	`/api/milestones/{id}`	MilestoneController
POST	`/api/milestones`	MilestoneController
PUT	`/api/milestones/{id}`	MilestoneController
POST	`/api/milestones/bulk-status`	MilestoneController
DELETE	`/api/milestones/{id}`	MilestoneController
POST	`/api/moderation/reports`	ModerationController
GET	`/api/moderation/reports/mine`	ModerationController
GET	`/api/moderation/reports`	ModerationController
GET	`/api/moderation/reports/queue`	ModerationController
PUT	`/api/moderation/reports/{reportId}`	ModerationController
GET	`/api/network/search`	NetworkController
GET	`/api/network/relationships`	NetworkController
GET	`/api/network/relationship-types`	NetworkController
POST	`/api/network/relationships`	NetworkController
POST	`/api/network/relationships/add-person`	NetworkController
PUT	`/api/network/relationships/{id}`	NetworkController
DELETE	`/api/network/relationships/{id}`	NetworkController
POST	`/api/network/relationships/bulk-import`	NetworkController
GET	`/api/network/relationships/bulk-import/template`	NetworkController
GET	`/api/network/circles`	NetworkController
POST	`/api/network/circles`	NetworkController
PUT	`/api/network/circles/{circleId}`	NetworkController
POST	`/api/network/circles/{circleId}/members`	NetworkController

DELETE	`/api/network/circles/{circleId}/members/{userId}`	NetworkController
POST	`/api/network/circles/{circleId}/admins/{memberUserId}`	NetworkController
DELETE	`/api/network/circles/{circleId}/admins/{memberUserId}`	NetworkController
GET	`/api/network/circles/{circleId}/posts`	NetworkController
GET	`/api/network/circles/unread-counts`	NetworkController
POST	`/api/network/circles/{circleId}/posts`	NetworkController
PUT	`/api/network/circles/{circleId}/posts/{postId}`	NetworkController
DELETE	`/api/network/circles/{circleId}/posts/{postId}/for-me`	NetworkController
DELETE	`/api/network/circles/{circleId}/posts/{postId}`	NetworkController
GET	`/api/network/circles/{circleId}/posts/search`	NetworkController
POST	`/api/network/circles/{circleId}/posts/{postId}/reaction`	NetworkController
GET	`/api/network/circles/{circleId}/posts/{postId}/attachment`	NetworkController
GET	`/api/network/messages/with/{otherUserId}`	NetworkController
GET	`/api/network/messages/conversations`	NetworkController
GET	`/api/network/messages/with/{otherUserId}/search`	NetworkController
POST	`/api/network/messages/with/{otherUserId}`	NetworkController
PUT	`/api/network/messages/with/{otherUserId}/{messageId}`	NetworkController
DELETE	`/api/network/messages/with/{otherUserId}/{messageId}/for-me`	NetworkController
DELETE	`/api/network/messages/with/{otherUserId}/{messageId}`	NetworkController
POST	`/api/network/messages/with/{otherUserId}/{messageId}/reaction`	NetworkController
GET	`/api/network/messages/with/{otherUserId}/{messageId}/attachment`	NetworkController
POST	`/api/network/calls`	NetworkController
GET	`/api/network/calls/incoming`	NetworkController
GET	`/api/network/calls/{callId}`	NetworkController
POST	`/api/network/calls/{callId}/accept`	NetworkController
POST	`/api/network/calls/{callId}/reject`	NetworkController
POST	`/api/network/calls/{callId}/end`	NetworkController
GET	`/api/network/broadcasts/preview`	NetworkController
POST	`/api/network/broadcasts`	NetworkController
GET	`/api/network/discover`	PeopleDiscoveryController
GET	`/api/notifications`	NotificationController
GET	`/api/notifications/unread-count`	NotificationController
POST	`/api/notifications/{id}/read`	NotificationController
POST	`/api/notifications/read-all`	NotificationController
GET	`/api/notifications/preferences`	NotificationController

PUT	`/api/notifications/preferences`	NotificationController
POST	`/api/notifications/devices`	NotificationController
DELETE	`/api/notifications/devices`	NotificationController
GET	`/api/notifications/unsubscribe/{token}`	NotificationController
GET	`/api/permissions`	PermissionController
GET	`/api/permissions/{id}`	PermissionController
POST	`/api/permissions`	PermissionController
PUT	`/api/permissions/{id}`	PermissionController
DELETE	`/api/permissions/{id}`	PermissionController
GET	`/api/people`	PersonController
GET	`/api/people/{id}`	PersonController
POST	`/api/people`	PersonController
PUT	`/api/people/{id}`	PersonController
DELETE	`/api/people/{id}`	PersonController
GET	`/api/people/{personId}/profile`	PersonProfileController
PUT	`/api/people/{personId}/record`	PersonProfileController
POST	`/api/people/{personId}/photo`	PersonProfileController
POST	`/api/people/{personId}/memories`	PersonProfileController
DELETE	`/api/people/{personId}/memories/{memoryId}`	PersonProfileController
GET	`/api/people/{personId}/memories/{memoryId}/media`	PersonProfileController
POST	`/api/network/presence/heartbeat`	PresenceController
GET	`/api/network/presence/direct/{userId}`	PresenceController
POST	`/api/network/presence/direct/{userId}/typing`	PresenceController
GET	`/api/network/presence/circles/{circleId}`	PresenceController
POST	`/api/network/presence/circles/{circleId}/typing`	PresenceController
GET	`/api/privacy/blocks`	PrivacyController
POST	`/api/privacy/blocks`	PrivacyController
DELETE	`/api/privacy/blocks/{userId}`	PrivacyController
GET	`/api/profile/me`	ProfileController
PUT	`/api/profile/me`	ProfileController
POST	`/api/profile/me/photo`	ProfileController
DELETE	`/api/profile/me/photo`	ProfileController
POST	`/api/profile/me/photos`	ProfileController
DELETE	`/api/profile/me/photos/{index}`	ProfileController
GET	`/api/profile/media/{name}`	ProfileController

GET	`/api/profile/me/media-usage`	ProfileController
GET	`/api/projects`	ProjectController
POST	`/api/projects`	ProjectController
PUT	`/api/projects/{id}`	ProjectController
GET	`/api/relationships`	RelationshipController
GET	`/api/relationships/{id}`	RelationshipController
POST	`/api/relationships`	RelationshipController
PUT	`/api/relationships/{id}`	RelationshipController
DELETE	`/api/relationships/{id}`	RelationshipController
GET	`/api/settings/me`	AppSettingsController
PUT	`/api/settings/me`	AppSettingsController
GET	`/api/social/feed`	SocialController
GET	`/api/social/saved`	SocialController
POST	`/api/social/posts`	SocialController
PUT	`/api/social/posts/{postId}`	SocialController
DELETE	`/api/social/posts/{postId}`	SocialController
POST	`/api/social/posts/{postId}/like`	SocialController
POST	`/api/social/posts/{postId}/save`	SocialController
POST	`/api/social/posts/{postId}/share`	SocialController
POST	`/api/social/posts/{postId}/comments`	SocialController
DELETE	`/api/social/posts/{postId}/comments/{commentId}`	SocialController
GET	`/api/social/posts/{postId}/media`	SocialController
GET	`/api/social/stories`	SocialController
POST	`/api/social/stories`	SocialController
POST	`/api/social/stories/{storyId}/view`	SocialController
DELETE	`/api/social/stories/{storyId}`	SocialController
GET	`/api/social/stories/{storyId}/media`	SocialController
GET	`/api/tasks`	TaskController
POST	`/api/tasks`	TaskController
PUT	`/api/tasks/{id}`	TaskController
GET	`/api/task-groups`	TaskGroupController
POST	`/api/task-groups`	TaskGroupController
GET	`/api/timeline/events`	TimelineController
DELETE	`/api/timeline/events/{key}`	TimelineController
PUT	`/api/timeline/events/{key}`	TimelineController

GET	`/api/trust/people`	TrustController
GET	`/api/trust/people/inbound`	TrustController
POST	`/api/trust/people`	TrustController
DELETE	`/api/trust/people/{userId}`	TrustController
GET	`/api/trust/following`	TrustController
GET	`/api/trust/followers`	TrustController
GET	`/api/trust/role-models`	TrustController
POST	`/api/trust/role-models/{userId}/follow`	TrustController
DELETE	`/api/trust/role-models/{userId}/follow`	TrustController
GET	`/api/trust/emergencies`	TrustController
POST	`/api/trust/emergencies`	TrustController
POST	`/api/trust/emergencies/{requestId}/decision`	TrustController
GET	`/api/trust/emergencies/{requestId}/documents`	TrustController
GET	`/api/trust/emergencies/{requestId}/documents/{postId}/media`	TrustController
GET	`/api/users`	UserController
GET	`/api/users/{id}`	UserController
POST	`/api/users`	UserController
PUT	`/api/users/{id}`	UserController
DELETE	`/api/users/{id}`	UserController
GET	`/api/uploads/policy`	UploadPolicyController
GET	`/api/admin/upload-levels`	UploadPolicyController
PUT	`/api/admin/upload-levels`	UploadPolicyController
PUT	`/api/admin/upload-levels/users/{id}`	UploadPolicyController
GET	`/api/admin/security`	SecurityReportsController
POST	`/api/admin/security/{id}/resolve`	SecurityReportsController
POST	`/api/admin/security/{id}/unblock`	SecurityReportsController

Python helper routes

Configured router prefix; source default uses `/api/v1`. These are internal helper contracts, not personalized Spring API routes.

- `GET /api/v1/health`
- `GET /api/v1/ready`
- `POST /api/v1/search/rank`
- `POST /api/v1/duplicates`
- `POST /api/v1/family/insights`
- `POST /api/v1/profiles/enrichment`
- `POST /api/v1/contacts/organize`

Appendix B — Database DDL reference

Database DDL reference

Snapshot 20 October 2026. Source-derived declaration inventory; consult migrations for final deployed types/defaults/constraints. Common migrations only; development seed data excluded. ALTER history is retained to avoid presenting original declarations as certified current catalog metadata.

abuse_reports

Created by V26__add_audit_and_abuse.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V26__add_audit_and_abuse.sql
`reporter_user_id`	`BIGINT NOT NULL`	V26__add_audit_and_abuse.sql
`reported_user_id`	`BIGINT`	V26__add_audit_and_abuse.sql
`entity_type`	`VARCHAR(40)`	V26__add_audit_and_abuse.sql
`entity_id`	`BIGINT`	V26__add_audit_and_abuse.sql
`reason`	`VARCHAR(40) NOT NULL`	V26__add_audit_and_abuse.sql
`details`	`VARCHAR(2000)`	V26__add_audit_and_abuse.sql
`status`	`VARCHAR(24) NOT NULL DEFAULT 'OPEN'`	V26__add_audit_and_abuse.sql
`moderator_user_id`	`BIGINT`	V26__add_audit_and_abuse.sql
`moderator_notes`	`VARCHAR(2000)`	V26__add_audit_and_abuse.sql
`created_at`	`TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`	V26__add_audit_and_abuse.sql
`updated_at`	`TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`	V26__add_audit_and_abuse.sql

Constraints and later ALTER history:

- `CONSTRAINT fk_abuse_reporter FOREIGN KEY (reporter_user_id) REFERENCES users(id) ON DELETE CASCADE`
- `CONSTRAINT fk_abuse_reported_user FOREIGN KEY (reported_user_id) REFERENCES users(id) ON DELETE SET NULL`

address_book_circle_contacts

Created by V59__private_address_book.sql.

Column	Declaration (with later additions)	Source
`contact_id`	`BIGINT NOT NULL REFERENCES address_book_contacts(id) ON DELETE CASCADE`	V59__private_address_book.sql
`circle_id`	`BIGINT NOT NULL REFERENCES circles(id) ON DELETE CASCADE`	V59__private_address_book.sql

Constraints and later ALTER history:

- `PRIMARY KEY(contact_id,circle_id)`

address_book_contacts

Created by V59__private_address_book.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V59__private_address_book.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V59__private_address_book.sql
`fingerprint`	`VARCHAR(64) NOT NULL`	V59__private_address_book.sql
`display_name`	`VARCHAR(200) NOT NULL`	V59__private_address_book.sql
`contact_data`	`TEXT NOT NULL`	V59__private_address_book.sql
`source`	`VARCHAR(20) NOT NULL`	V59__private_address_book.sql
`relationship_id`	`BIGINT REFERENCES relationships(id) ON DELETE SET NULL`	V59__private_address_book.sql
`circles_reviewed`	`BOOLEAN NOT NULL DEFAULT FALSE`	V59__private_address_book.sql
`updated_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V59__private_address_book.sql

Constraints and later ALTER history:

- `UNIQUE(user_id, fingerprint)`

admin_mfa

Created by V55__verified_phone_and_admin_mfa.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE`	V55__verified_phone_and_admin_mfa.sql
`encrypted_secret`	`VARCHAR(512) NOT NULL`	V55__verified_phone_and_admin_mfa.sql
`last_counter`	`BIGINT NOT NULL DEFAULT -1`	V55__verified_phone_and_admin_mfa.sql

ai_action_approvals

Created by V50__add_ai_action_approvals.sql.

Column	Declaration (with later additions)	Source
`id`	`UUID PRIMARY KEY`	V50__add_ai_action_approvals.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V50__add_ai_action_approvals.sql

capability	VARCHAR(80) NOT NULL	V50__add_ai_action_approvals.sql
action_level	VARCHAR(2) NOT NULL	V50__add_ai_action_approvals.sql
title	VARCHAR(120) NOT NULL	V50__add_ai_action_approvals.sql
summary	VARCHAR(500) NOT NULL	V50__add_ai_action_approvals.sql
resource_type	VARCHAR(80)	V50__add_ai_action_approvals.sql
resource_id	VARCHAR(120)	V50__add_ai_action_approvals.sql
action_fingerprint	VARCHAR(64) NOT NULL	V50__add_ai_action_approvals.sql
pending_fingerprint	VARCHAR(64)	V50__add_ai_action_approvals.sql
status	VARCHAR(20) NOT NULL DEFAULT 'PENDING'	V50__add_ai_action_approvals.sql
decision_reason	VARCHAR(240)	V50__add_ai_action_approvals.sql
requested_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V50__add_ai_action_approvals.sql
expires_at	TIMESTAMP WITH TIME ZONE NOT NULL	V50__add_ai_action_approvals.sql
decided_at	TIMESTAMP WITH TIME ZONE	V50__add_ai_action_approvals.sql
version	BIGINT NOT NULL DEFAULT 0	V50__add_ai_action_approvals.sql
authorization_expires_at	TIMESTAMP WITH TIME ZONE	V51__add_ai_action_execution_guard.sql

Constraints and later ALTER history:

- CONSTRAINT chk_ai_approval_level CHECK (action_level IN ('L2', 'L3'))
- CONSTRAINT chk_ai_action_approval_status CHECK (status IN ('PENDING', 'APPROVED', 'REJECTED', 'EXPIRED', 'CANCELLED'))
- CONSTRAINT chk_ai_action_approval_expiry CHECK (expires_at > requested_at)

ai_action_events

Created by V48__add_ai_action_ledger.sql.

Column	Declaration (with later additions)	Source
id	BIGSERIAL PRIMARY KEY	V48__add_ai_action_ledger.sql
request_id	UUID NOT NULL UNIQUE	V48__add_ai_action_ledger.sql
user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V48__add_ai_action_ledger.sql
capability	VARCHAR(80) NOT NULL	V48__add_ai_action_ledger.sql
action_level	VARCHAR(2) NOT NULL	V48__add_ai_action_ledger.sql
purpose	VARCHAR(240) NOT NULL	V48__add_ai_action_ledger.sql
consent_granted	BOOLEAN NOT NULL DEFAULT FALSE	V48__add_ai_action_ledger.sql
approval_state	VARCHAR(20) NOT NULL	V48__add_ai_action_ledger.sql
status	VARCHAR(20) NOT NULL	V48__add_ai_action_ledger.sql
failure_code	VARCHAR(80)	V48__add_ai_action_ledger.sql

`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V48__add_ai_action_ledger.sql
`completed_at`	`TIMESTAMP WITH TIME ZONE`	V48__add_ai_action_ledger.sql

Constraints and later ALTER history:

- `CONSTRAINT chk_ai_action_level CHECK (action_level IN ('L0', 'L1', 'L2', 'L3', 'L4'))`
- `CONSTRAINT chk_ai_approval_state CHECK (approval_state IN ('NOT_REQUIRED', 'REQUIRED', 'APPROVED', 'REJECTED'))`
- `CONSTRAINT chk_ai_action_status CHECK (status IN ('STARTED', 'SUCCEEDED', 'FAILED', 'REJECTED'))`

ai_action_executions

Created by V51__add_ai_action_execution_guard.sql.

Column	Declaration (with later additions)	Source
`id`	`UUID PRIMARY KEY`	V51__add_ai_action_execution_guard.sql
`approval_id`	`UUID NOT NULL UNIQUE REFERENCES ai_action_approvals(id) ON DELETE CASCADE`	V51__add_ai_action_execution_guard.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V51__add_ai_action_execution_guard.sql
`idempotency_key_hash`	`VARCHAR(64) NOT NULL`	V51__add_ai_action_execution_guard.sql
`status`	`VARCHAR(20) NOT NULL`	V51__add_ai_action_execution_guard.sql
`failure_code`	`VARCHAR(80)`	V51__add_ai_action_execution_guard.sql
`result_reference`	`VARCHAR(240)`	V51__add_ai_action_execution_guard.sql
`started_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V51__add_ai_action_execution_guard.sql
`completed_at`	`TIMESTAMP WITH TIME ZONE`	V51__add_ai_action_execution_guard.sql
`version`	`BIGINT NOT NULL DEFAULT 0`	V51__add_ai_action_execution_guard.sql

Constraints and later ALTER history:

- `CONSTRAINT chk_ai_action_execution_status CHECK ( status IN ('RUNNING', 'SUCCEEDED', 'FAILED'))`

ai_user_preferences

Created by V49__add_ai_user_preferences.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE`	V49__add_ai_user_preferences.sql

`ai_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V49__add_ai_user_preferences.sql
`allow_sensitive_data`	`BOOLEAN NOT NULL DEFAULT FALSE`	V49__add_ai_user_preferences.sql
`allow_personalization`	`BOOLEAN NOT NULL DEFAULT FALSE`	V49__add_ai_user_preferences.sql
`activity_retention_days`	`INTEGER NOT NULL DEFAULT 90`	V49__add_ai_user_preferences.sql
`updated_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V49__add_ai_user_preferences.sql

Constraints and later ALTER history:

- `CONSTRAINT chk_ai_activity_retention CHECK (activity_retention_days BETWEEN 0 AND 365)`

app_settings

Created by V57__application_settings.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE`	V57__application_settings.sql
`discoverable`	`BOOLEAN NOT NULL DEFAULT FALSE`	V57__application_settings.sql
`share_interests`	`BOOLEAN NOT NULL DEFAULT FALSE`	V57__application_settings.sql
`settings_json`	`TEXT NOT NULL`	V57__application_settings.sql
`updated_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V57__application_settings.sql

audit_events

Created by V26__add_audit_and_abuse.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V26__add_audit_and_abuse.sql
`actor_user_id`	`BIGINT`	V26__add_audit_and_abuse.sql
`action`	`VARCHAR(20) NOT NULL`	V26__add_audit_and_abuse.sql
`request_path`	`VARCHAR(700) NOT NULL`	V26__add_audit_and_abuse.sql
`resource_type`	`VARCHAR(80)`	V26__add_audit_and_abuse.sql
`resource_id`	`VARCHAR(120)`	V26__add_audit_and_abuse.sql
`response_status`	`INTEGER NOT NULL`	V26__add_audit_and_abuse.sql
`ip_address`	`VARCHAR(64)`	V26__add_audit_and_abuse.sql
`user_agent`	`VARCHAR(500)`	V26__add_audit_and_abuse.sql
`occurred_at`	`TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`	V26__add_audit_and_abuse.sql

auth_tokens

Created by V5__add_auth_tokens_table.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V5__add_auth_tokens_table.sql
`token`	`VARCHAR(2048) NOT NULL UNIQUE`	V5__add_auth_tokens_table.sql
`user_id`	`BIGINT`	V5__add_auth_tokens_table.sql
`token_type`	`VARCHAR(32) NOT NULL`	V5__add_auth_tokens_table.sql
`expires_at`	`TIMESTAMP NOT NULL`	V5__add_auth_tokens_table.sql
`session_id`	`VARCHAR(36)`	V54__revocable_sessions.sql

Constraints and later ALTER history:

- `V5__add_auth_tokens_table.sql: ADD CONSTRAINT fk_auth_tokens_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE`

circle_admins

Created by V14__add_circle_admins.sql.

Column	Declaration (with later additions)	Source
`circle_id`	`BIGINT NOT NULL`	V14__add_circle_admins.sql
`user_id`	`BIGINT NOT NULL`	V14__add_circle_admins.sql

Constraints and later ALTER history:

- `PRIMARY KEY (circle_id, user_id)`
- `CONSTRAINT fk_circle_admin_circle FOREIGN KEY (circle_id) REFERENCES circles(id) ON DELETE CASCADE`
- `CONSTRAINT fk_circle_admin_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE`

circle_members

Created by V11__add_social_network.sql.

Column	Declaration (with later additions)	Source
`circle_id`	`BIGINT NOT NULL`	V11__add_social_network.sql
`user_id`	`BIGINT NOT NULL`	V11__add_social_network.sql

Constraints and later ALTER history:

- `PRIMARY KEY (circle_id, user_id)`
- `CONSTRAINT fk_circle_member_circle FOREIGN KEY (circle_id) REFERENCES circles(id) ON DELETE CASCADE`
- `CONSTRAINT fk_circle_member_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE`

circle_post_hidden

Created by V63__private_message_deletions.sql.

Column	Declaration (with later additions)	Source
post_id	BIGINT NOT NULL REFERENCES circle_posts(id) ON DELETE CASCADE	V63__private_message_deletions.sql
user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V63__private_message_deletions.sql

Constraints and later ALTER history:

- PRIMARY KEY(post_id,user_id)

circle_post_pins

Created by V64__circle_message_flags.sql.

Column	Declaration (with later additions)	Source
post_id	BIGINT NOT NULL REFERENCES circle_posts(id) ON DELETE CASCADE	V64__circle_message_flags.sql
user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V64__circle_message_flags.sql

Constraints and later ALTER history:

- PRIMARY KEY(post_id,user_id)

circle_post_reactions

Created by V35__add_circle_message_reactions.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V35__add_circle_message_reactions.sql
post_id	BIGINT NOT NULL REFERENCES circle_posts(id) ON DELETE CASCADE	V35__add_circle_message_reactions.sql
user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V35__add_circle_message_reactions.sql
emoji	VARCHAR(16) NOT NULL	V35__add_circle_message_reactions.sql
created_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V35__add_circle_message_reactions.sql

Constraints and later ALTER history:

- CONSTRAINT uq_circle_post_reaction UNIQUE(post_id,user_id)

circle_post_reads

Created by V36__add_circle_message_reads.sql.

Column	Declaration (with later additions)	Source

`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V36__add_circle_message_reads.sql
`post_id`	`BIGINT NOT NULL REFERENCES circle_posts(id) ON DELETE CASCADE`	V36__add_circle_message_reads.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V36__add_circle_message_reads.sql
`read_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V36__add_circle_message_reads.sql

Constraints and later ALTER history:

- `CONSTRAINT uq_circle_post_read UNIQUE(post_id,user_id)`

circle_post_stars

Created by V64__circle_message_flags.sql.

Column	Declaration (with later additions)	Source
`post_id`	`BIGINT NOT NULL REFERENCES circle_posts(id) ON DELETE CASCADE`	V64__circle_message_flags.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V64__circle_message_flags.sql

Constraints and later ALTER history:

- `PRIMARY KEY(post_id,user_id)`

circle_posts

Created by V21__add_circle_conversations.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V21__add_circle_conversations.sql
`circle_id`	`BIGINT NOT NULL`	V21__add_circle_conversations.sql
`author_user_id`	`BIGINT NOT NULL`	V21__add_circle_conversations.sql
`parent_post_id`	`BIGINT`	V21__add_circle_conversations.sql
`message`	`VARCHAR(4000) NOT NULL DEFAULT ''`	V21__add_circle_conversations.sql
`attachment_key`	`VARCHAR(255)`	V21__add_circle_conversations.sql
`attachment_name`	`VARCHAR(255)`	V21__add_circle_conversations.sql
`attachment_type`	`VARCHAR(255)`	V21__add_circle_conversations.sql
`attachment_size`	`BIGINT`	V21__add_circle_conversations.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V21__add_circle_conversations.sql
`edited_at`	`TIMESTAMP NULL`	V34__add_circle_message_lifecycle.sql
`deleted_at`	`TIMESTAMP NULL`	V34__add_circle_message_lifecycle.sql

forwarded	BOOLEAN NOT NULL DEFAULT FALSE	V64__circle_message_flags.sql

Constraints and later ALTER history:

- CONSTRAINT fk_circle_post_circle FOREIGN KEY (circle_id) REFERENCES circles(id) ON DELETE CASCADE
- CONSTRAINT fk_circle_post_author FOREIGN KEY (author_user_id) REFERENCES users(id)
- CONSTRAINT fk_circle_post_parent FOREIGN KEY (parent_post_id) REFERENCES circle_posts(id) ON DELETE CASCADE

circles

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V1__create_core_tables.sql
name	VARCHAR(255) NOT NULL UNIQUE	V1__create_core_tables.sql
description	VARCHAR(1024) NOT NULL	V1__create_core_tables.sql
owner_user_id	BIGINT	V11__add_social_network.sql
posting_permission	VARCHAR(32) NOT NULL DEFAULT 'ALL_MEMBERS'	V21__add_circle_conversations.sql

Constraints and later ALTER history:

- V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_circles_name_not_blank CHECK (CHAR_LENGTH(TRIM(name)) > 0)
- V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_circles_description_not_blank CHECK (CHAR_LENGTH(TRIM(description)) > 0)
- V11__add_social_network.sql: ADD CONSTRAINT fk_circle_owner FOREIGN KEY (owner_user_id) REFERENCES users(id) ON DELETE CASCADE

direct_calls

Created by V23__add_direct_calls.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V23__add_direct_calls.sql
caller_user_id	BIGINT NOT NULL	V23__add_direct_calls.sql
recipient_user_id	BIGINT NOT NULL	V23__add_direct_calls.sql
call_type	VARCHAR(16) NOT NULL	V23__add_direct_calls.sql
status	VARCHAR(24) NOT NULL DEFAULT 'RINGING'	V23__add_direct_calls.sql
offer_sdp	TEXT NOT NULL	V23__add_direct_calls.sql
answer_sdp	TEXT	V23__add_direct_calls.sql
created_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V23__add_direct_calls.sql
updated_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V23__add_direct_calls.sql

Constraints and later ALTER history:

- `CONSTRAINT fk_direct_call_caller FOREIGN KEY (caller_user_id) REFERENCES users(id)`
- `CONSTRAINT fk_direct_call_recipient FOREIGN KEY (recipient_user_id) REFERENCES users(id)`
- `CONSTRAINT chk_direct_call_participants CHECK (caller_user_id <> recipient_user_id)`
- `CONSTRAINT chk_direct_call_type CHECK (call_type IN ('AUDIO','VIDEO'))`
- `CONSTRAINT chk_direct_call_status CHECK (status IN ('RINGING','ACCEPTED','REJECTED','ENDED'))`

direct_chat_pins

Created by V58__message_personalization.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V58__message_personalization.sql
`other_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V58__message_personalization.sql

Constraints and later ALTER history:

- `PRIMARY KEY(user_id, other_user_id)`

direct_chat_settings

Created by V61__direct_chat_settings.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V61__direct_chat_settings.sql
`other_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V61__direct_chat_settings.sql
`archived`	`BOOLEAN NOT NULL DEFAULT FALSE`	V61__direct_chat_settings.sql
`muted`	`BOOLEAN NOT NULL DEFAULT FALSE`	V61__direct_chat_settings.sql
`marked_unread`	`BOOLEAN NOT NULL DEFAULT FALSE`	V61__direct_chat_settings.sql

Constraints and later ALTER history:

- `PRIMARY KEY (user_id, other_user_id)`
- `CHECK (user_id <> other_user_id)`

direct_message_flags

Created by V58__message_personalization.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V58__message_personalization.sql
`message_id`	`BIGINT NOT NULL REFERENCES direct_messages(id) ON DELETE CASCADE`	V58__message_personalization.sql
`starred`	`BOOLEAN NOT NULL DEFAULT FALSE`	V58__message_personalization.sql
`pinned`	`BOOLEAN NOT NULL DEFAULT FALSE`	V58__message_personalization.sql

Constraints and later ALTER history:

- `PRIMARY KEY (user_id, message_id)`

direct_message_reactions

Created by V32__add_direct_message_interactions.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V32__add_direct_message_interactions.sql
`message_id`	`BIGINT NOT NULL REFERENCES direct_messages(id) ON DELETE CASCADE`	V32__add_direct_message_interactions.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V32__add_direct_message_interactions.sql
`emoji`	`VARCHAR(16) NOT NULL`	V32__add_direct_message_interactions.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V32__add_direct_message_interactions.sql

Constraints and later ALTER history:

- `CONSTRAINT uq_direct_message_reaction UNIQUE(message_id,user_id)`

direct_messages

Created by V22__add_direct_messages.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V22__add_direct_messages.sql
`sender_user_id`	`BIGINT NOT NULL`	V22__add_direct_messages.sql
`recipient_user_id`	`BIGINT NOT NULL`	V22__add_direct_messages.sql
`message`	`VARCHAR(4000) NOT NULL DEFAULT ''`	V22__add_direct_messages.sql
`attachment_key`	`VARCHAR(255)`	V22__add_direct_messages.sql
`attachment_name`	`VARCHAR(255)`	V22__add_direct_messages.sql
`attachment_type`	`VARCHAR(255)`	V22__add_direct_messages.sql
`attachment_size`	`BIGINT`	V22__add_direct_messages.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V22__add_direct_messages.sql
`delivered_at`	`TIMESTAMP WITH TIME ZONE`	V30__add_direct_message_receipts.sql
`read_at`	`TIMESTAMP WITH TIME ZONE`	V30__add_direct_message_receipts.sql
`reply_to_message_id`	`BIGINT REFERENCES direct_messages(id) ON DELETE SET NULL`	V32__add_direct_message_interactions.sql

`edited_at`	`TIMESTAMP WITH TIME ZONE`	V32__add_direct_message_interactions.sql
`deleted_at`	`TIMESTAMP WITH TIME ZONE`	V32__add_direct_message_interactions.sql
`forwarded`	`BOOLEAN NOT NULL DEFAULT FALSE`	V62__forwarded_direct_messages.sql
`sender_hidden`	`BOOLEAN NOT NULL DEFAULT FALSE`	V63__private_message_deletions.sql
`recipient_hidden`	`BOOLEAN NOT NULL DEFAULT FALSE`	V63__private_message_deletions.sql

Constraints and later ALTER history:

- `CONSTRAINT fk_direct_message_sender FOREIGN KEY (sender_user_id) REFERENCES users(id)`
- `CONSTRAINT fk_direct_message_recipient FOREIGN KEY (recipient_user_id) REFERENCES users(id)`
- `CONSTRAINT chk_direct_message_participants CHECK (sender_user_id <> recipient_user_id)`

emergency_access_approvals

Created by V44__add_trusted_people.sql.

Column	Declaration (with later additions)	Source
`request_id`	`BIGINT NOT NULL REFERENCES emergency_access_requests(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`approver_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`approved`	`BOOLEAN NOT NULL`	V44__add_trusted_people.sql
`decided_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V44__add_trusted_people.sql

Constraints and later ALTER history:

- `PRIMARY KEY (request_id, approver_user_id)`

emergency_access_events

Created by V44__add_trusted_people.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V44__add_trusted_people.sql
`request_id`	`BIGINT NOT NULL REFERENCES emergency_access_requests(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`actor_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`action`	`VARCHAR(40) NOT NULL`	V44__add_trusted_people.sql
`resource_id`	`BIGINT`	V44__add_trusted_people.sql

`occurred_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V44__add_trusted_people.sql

emergency_access_requests

Created by V44__add_trusted_people.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V44__add_trusted_people.sql
`owner_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`requester_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`reason`	`VARCHAR(1000) NOT NULL`	V44__add_trusted_people.sql
`status`	`VARCHAR(20) NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING','APPROVED','REJECTED','EXPIRED'))`	V44__add_trusted_people.sql
`required_approvals`	`INTEGER NOT NULL DEFAULT 2`	V44__add_trusted_people.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V44__add_trusted_people.sql
`expires_at`	`TIMESTAMP WITH TIME ZONE NOT NULL`	V44__add_trusted_people.sql
`granted_until`	`TIMESTAMP WITH TIME ZONE`	V44__add_trusted_people.sql

financial_transactions

Created by V45__add_financial_tracking.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V45__add_financial_tracking.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V45__add_financial_tracking.sql
`amount`	`DECIMAL(15,2) NOT NULL`	V45__add_financial_tracking.sql
`direction`	`VARCHAR(12) NOT NULL`	V45__add_financial_tracking.sql
`category`	`VARCHAR(32) NOT NULL`	V45__add_financial_tracking.sql
`merchant`	`VARCHAR(160)`	V45__add_financial_tracking.sql
`description`	`VARCHAR(500)`	V45__add_financial_tracking.sql

`occurred_at`	`TIMESTAMP WITH TIME ZONE NOT NULL`	V45__add_financial_tracking.sql
`source`	`VARCHAR(20) NOT NULL`	V45__add_financial_tracking.sql
`source_fingerprint`	`VARCHAR(64)`	V45__add_financial_tracking.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V45__add_financial_tracking.sql

Constraints and later ALTER history:

- `CONSTRAINT financial_direction_check CHECK (direction IN ('EXPENSE','INCOME'))`
- `CONSTRAINT financial_amount_check CHECK (amount > 0)`
- `CONSTRAINT financial_source_check CHECK (source IN ('MANUAL','SMS','IMPORT'))`
- `CONSTRAINT financial_source_unique UNIQUE (user_id, source_fingerprint)`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN merchant TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN description TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN amount TYPE TEXT`

health_measurements

Created by V46__add_health_reports.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V46__add_health_reports.sql
`report_id`	`BIGINT NOT NULL REFERENCES health_reports(id) ON DELETE CASCADE`	V46__add_health_reports.sql
`metric_name`	`VARCHAR(120) NOT NULL`	V46__add_health_reports.sql
`measured_value`	`DECIMAL(18,4) NOT NULL`	V46__add_health_reports.sql
`unit`	`VARCHAR(40) NOT NULL`	V46__add_health_reports.sql
`reference_min`	`DECIMAL(18,4)`	V46__add_health_reports.sql
`reference_max`	`DECIMAL(18,4)`	V46__add_health_reports.sql
`suggestion`	`VARCHAR(500)`	V47__add_health_measurement_suggestions.sql

Constraints and later ALTER history:

- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN metric_name TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN unit TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN suggestion TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN measured_value TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN reference_min TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN reference_max TYPE TEXT`

health_reports

Created by V46__add_health_reports.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V46__add_health_reports.sql
user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V46__add_health_reports.sql
source_post_id	BIGINT REFERENCES social_posts(id) ON DELETE SET NULL	V46__add_health_reports.sql
report_name	VARCHAR(200) NOT NULL	V46__add_health_reports.sql
laboratory	VARCHAR(200)	V46__add_health_reports.sql
collected_on	DATE NOT NULL	V46__add_health_reports.sql
notes	VARCHAR(1000)	V46__add_health_reports.sql
created_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V46__add_health_reports.sql

Constraints and later ALTER history:

- V56__encrypted_sensitive_fields.sql: ALTER COLUMN report_name TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN laboratory TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN notes TYPE TEXT

help_ai_settings

Created by V65__help_answer_memory.sql.

Column	Declaration (with later additions)	Source
id	INTEGER PRIMARY KEY CHECK (id = 1)	V65__help_answer_memory.sql
enhanced_enabled	BOOLEAN NOT NULL DEFAULT FALSE	V65__help_answer_memory.sql

help_exchanges

Created by V65__help_answer_memory.sql.

Column	Declaration (with later additions)	Source
id	VARCHAR(36) PRIMARY KEY	V65__help_answer_memory.sql
user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V65__help_answer_memory.sql
cache_key	VARCHAR(64) NOT NULL	V65__help_answer_memory.sql
tier	VARCHAR(16) NOT NULL	V65__help_answer_memory.sql
mode	VARCHAR(16) NOT NULL	V65__help_answer_memory.sql
model	VARCHAR(80) NOT NULL	V65__help_answer_memory.sql
input_tokens	INTEGER NOT NULL DEFAULT 0	V65__help_answer_memory.sql
output_tokens	INTEGER NOT NULL DEFAULT 0	V65__help_answer_memory.sql
payload	TEXT NOT NULL	V65__help_answer_memory.sql
created_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V65__help_answer_memory.sql

`expires_at`	`TIMESTAMP WITH TIME ZONE NOT NULL`	V65__help_answer_memory.sql

help_shared_answers

Created by V65__help_answer_memory.sql.

Column	Declaration (with later additions)	Source
`question_key`	`VARCHAR(64) PRIMARY KEY`	V65__help_answer_memory.sql
`knowledge_version`	`VARCHAR(64) NOT NULL`	V65__help_answer_memory.sql
`payload`	`TEXT NOT NULL`	V65__help_answer_memory.sql
`approved_by`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V65__help_answer_memory.sql
`expires_at`	`TIMESTAMP WITH TIME ZONE NOT NULL`	V65__help_answer_memory.sql

media_assets

Created by V25__add_media_assets.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V25__add_media_assets.sql
`owner_user_id`	`BIGINT NOT NULL`	V25__add_media_assets.sql
`storage_key`	`VARCHAR(700) NOT NULL UNIQUE`	V25__add_media_assets.sql
`category`	`VARCHAR(40) NOT NULL`	V25__add_media_assets.sql
`original_name`	`VARCHAR(255) NOT NULL`	V25__add_media_assets.sql
`content_type`	`VARCHAR(150) NOT NULL`	V25__add_media_assets.sql
`size_bytes`	`BIGINT NOT NULL`	V25__add_media_assets.sql
`sha256`	`VARCHAR(64) NOT NULL`	V25__add_media_assets.sql
`scan_status`	`VARCHAR(24) NOT NULL DEFAULT 'CLEAN'`	V25__add_media_assets.sql
`thumbnail_key`	`VARCHAR(700)`	V25__add_media_assets.sql
`expires_at`	`TIMESTAMP`	V25__add_media_assets.sql
`deleted_at`	`TIMESTAMP`	V25__add_media_assets.sql
`created_at`	`TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`	V25__add_media_assets.sql

Constraints and later ALTER history:

- `CONSTRAINT fk_media_asset_owner FOREIGN KEY (owner_user_id) REFERENCES users(id) ON DELETE CASCADE`
- `CONSTRAINT ck_media_asset_size CHECK (size_bytes >= 0)`

message_preferences

Created by V58__message_personalization.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE`	V58__message_personalization.sql
`read_receipts`	`BOOLEAN NOT NULL DEFAULT TRUE`	V58__message_personalization.sql

milestones

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V1__create_core_tables.sql
`name`	`VARCHAR(255) NOT NULL`	V1__create_core_tables.sql
`description`	`VARCHAR(1024) NOT NULL`	V1__create_core_tables.sql
`status`	`VARCHAR(255) NOT NULL`	V1__create_core_tables.sql
`project_id`	`BIGINT`	V1__create_core_tables.sql
`due_date`	`DATE`	V1__create_core_tables.sql
`blocked_reason`	`VARCHAR(1024)`	V1__create_core_tables.sql

Constraints and later ALTER history:

- `V3__add_indexes_and_foreign_keys.sql: ADD CONSTRAINT fk_milestones_project FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE SET NULL`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_milestones_name_not_blank CHECK (CHAR_LENGTH(TRIM(name)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_milestones_description_not_blank CHECK (CHAR_LENGTH(TRIM(description)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_milestones_status_allowed CHECK (status IN ('Planned', 'In Progress', 'Blocked', 'Completed'))`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_milestones_blocked_reason_semantics CHECK ( (status = 'Blocked' AND blocked_reason IS NOT NULL AND CHAR_LENGTH(TRIM(blocked_reason)) > 0) OR (status <> 'Blocked' AND (blocked_reason IS NULL OR CHAR_LENGTH(TRIM(blocked_reason)) = 0)) )`

notification_deliveries

Created by V24__add_notifications.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V24__add_notifications.sql
`notification_id`	`BIGINT NOT NULL`	V24__add_notifications.sql
`channel`	`VARCHAR(20) NOT NULL`	V24__add_notifications.sql
`destination`	`VARCHAR(512) NOT NULL`	V24__add_notifications.sql
`status`	`VARCHAR(24) NOT NULL DEFAULT 'PENDING'`	V24__add_notifications.sql

attempts	INTEGER NOT NULL DEFAULT 0	V24__add_notifications.sql
next_attempt_at	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V24__add_notifications.sql
provider_message_id	VARCHAR(255)	V24__add_notifications.sql
last_error	VARCHAR(1000)	V24__add_notifications.sql
sent_at	TIMESTAMP	V24__add_notifications.sql
created_at	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V24__add_notifications.sql
updated_at	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V24__add_notifications.sql
lease_until	TIMESTAMP WITH TIME ZONE	V52__scaling_indexes_and_delivery_leases.sql
claim_token	VARCHAR(36)	V52__scaling_indexes_and_delivery_leases.sql

Constraints and later ALTER history:

- CONSTRAINT fk_notification_delivery_notification FOREIGN KEY (notification_id) REFERENCES notifications(id) ON DELETE CASCADE
- CONSTRAINT uq_notification_delivery UNIQUE (notification_id, channel, destination)

notification_device_tokens

Created by V24__add_notifications.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V24__add_notifications.sql
user_id	BIGINT NOT NULL	V24__add_notifications.sql
token	VARCHAR(512) NOT NULL UNIQUE	V24__add_notifications.sql
platform	VARCHAR(20) NOT NULL	V24__add_notifications.sql
enabled	BOOLEAN NOT NULL DEFAULT TRUE	V24__add_notifications.sql
created_at	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V24__add_notifications.sql
updated_at	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V24__add_notifications.sql

Constraints and later ALTER history:

- CONSTRAINT fk_notification_device_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

notification_preferences

Created by V24__add_notifications.sql.

Column	Declaration (with later additions)	Source
user_id	BIGINT PRIMARY KEY	V24__add_notifications.sql
email_enabled	BOOLEAN NOT NULL DEFAULT TRUE	V24__add_notifications.sql

`sms_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V24__add_notifications.sql
`push_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V24__add_notifications.sql
`messages_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V24__add_notifications.sql
`circles_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V24__add_notifications.sql
`relationships_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V24__add_notifications.sql
`calls_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V24__add_notifications.sql
`invitations_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V24__add_notifications.sql
`unsubscribe_token`	`VARCHAR(96) NOT NULL UNIQUE`	V24__add_notifications.sql
`updated_at`	`TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`	V24__add_notifications.sql
`social_enabled`	`BOOLEAN NOT NULL DEFAULT TRUE`	V33__add_social_notification_preference.sql

Constraints and later ALTER history:

- `CONSTRAINT fk_notification_preferences_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE`

notifications

Created by V24__add_notifications.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V24__add_notifications.sql
`user_id`	`BIGINT NOT NULL`	V24__add_notifications.sql
`type`	`VARCHAR(40) NOT NULL`	V24__add_notifications.sql
`title`	`VARCHAR(180) NOT NULL`	V24__add_notifications.sql
`body`	`VARCHAR(2000) NOT NULL`	V24__add_notifications.sql
`action_url`	`VARCHAR(500)`	V24__add_notifications.sql
`entity_type`	`VARCHAR(40)`	V24__add_notifications.sql
`entity_id`	`BIGINT`	V24__add_notifications.sql
`read_at`	`TIMESTAMP`	V24__add_notifications.sql
`created_at`	`TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP`	V24__add_notifications.sql

Constraints and later ALTER history:

- `CONSTRAINT fk_notification_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE`

people

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source

`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V1__create_core_tables.sql
`full_name`	`VARCHAR(255) NOT NULL`	V1__create_core_tables.sql
`email`	`VARCHAR(255) NOT NULL`	V1__create_core_tables.sql
`gender`	`VARCHAR(32) NOT NULL DEFAULT 'Unspecified'`	V10__add_person_gender.sql

Constraints and later ALTER history:

- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_people_full_name_not_blank CHECK (CHAR_LENGTH(TRIM(full_name)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_people_email_not_blank CHECK (CHAR_LENGTH(TRIM(email)) > 0)`

permissions

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V1__create_core_tables.sql
`name`	`VARCHAR(255) NOT NULL UNIQUE`	V1__create_core_tables.sql
`description`	`VARCHAR(1024) NOT NULL`	V1__create_core_tables.sql

Constraints and later ALTER history:

- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_permissions_name_not_blank CHECK (CHAR_LENGTH(TRIM(name)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_permissions_description_not_blank CHECK (CHAR_LENGTH(TRIM(description)) > 0)`

person_memories

Created by V39__add_person_records_and_memories.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V39__add_person_records_and_memories.sql
`owner_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V39__add_person_records_and_memories.sql
`person_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V39__add_person_records_and_memories.sql
`title`	`VARCHAR(255)`	V39__add_person_records_and_memories.sql
`note`	`TEXT`	V39__add_person_records_and_memories.sql
`media_key`	`VARCHAR(512)`	V39__add_person_records_and_memories.sql
`media_name`	`VARCHAR(255)`	V39__add_person_records_and_memories.sql

media_type	VARCHAR(160)	V39__add_person_records_and_memories.sql
media_size	BIGINT	V39__add_person_records_and_memories.sql
created_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V39__add_person_records_and_memories.sql

person_records

Created by V39__add_person_records_and_memories.sql.

Column	Declaration (with later additions)	Source
owner_user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V39__add_person_records_and_memories.sql
person_user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V39__add_person_records_and_memories.sql
nickname	VARCHAR(255)	V39__add_person_records_and_memories.sql
contact_phone	VARCHAR(40)	V39__add_person_records_and_memories.sql
contact_email	VARCHAR(255)	V39__add_person_records_and_memories.sql
address	TEXT	V39__add_person_records_and_memories.sql
city	VARCHAR(120)	V39__add_person_records_and_memories.sql
country	VARCHAR(120)	V39__add_person_records_and_memories.sql
occupation	VARCHAR(255)	V39__add_person_records_and_memories.sql
important_dates	TEXT	V39__add_person_records_and_memories.sql
notes	TEXT	V39__add_person_records_and_memories.sql
updated_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V39__add_person_records_and_memories.sql
date_of_birth	VARCHAR(10)	V41__add_structured_person_dates.sql
marriage_date	VARCHAR(10)	V41__add_structured_person_dates.sql
date_of_death	VARCHAR(10)	V41__add_structured_person_dates.sql

Constraints and later ALTER history:

- PRIMARY KEY (owner_user_id, person_user_id)
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN nickname TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN contact_phone TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN contact_email TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN address TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN city TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN country TYPE TEXT

- V56__encrypted_sensitive_fields.sql: ALTER COLUMN occupation TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN date_of_birth TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN marriage_date TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN date_of_death TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN important_dates TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN notes TYPE TEXT

projects

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V1__create_core_tables.sql
name	VARCHAR(255) NOT NULL	V1__create_core_tables.sql
description	VARCHAR(1024) NOT NULL	V1__create_core_tables.sql
status	VARCHAR(255) NOT NULL	V1__create_core_tables.sql

Constraints and later ALTER history:

- V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_projects_name_not_blank CHECK (CHAR_LENGTH(TRIM(name)) > 0)
- V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_projects_description_not_blank CHECK (CHAR_LENGTH(TRIM(description)) > 0)
- V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_projects_status_allowed CHECK (status IN ('Active', 'Planning', 'Completed'))

relationship_broadcasts

Created by V27__add_relationship_broadcasts.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V27__add_relationship_broadcasts.sql
sender_user_id	BIGINT NOT NULL	V27__add_relationship_broadcasts.sql
audience_type	VARCHAR(24) NOT NULL	V27__add_relationship_broadcasts.sql
anchor_user_id	BIGINT	V27__add_relationship_broadcasts.sql
location_query	VARCHAR(255)	V27__add_relationship_broadcasts.sql
message_preview	VARCHAR(500)	V27__add_relationship_broadcasts.sql
recipient_count	INTEGER NOT NULL DEFAULT 0	V27__add_relationship_broadcasts.sql
failed_count	INTEGER NOT NULL DEFAULT 0	V27__add_relationship_broadcasts.sql
created_at	TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP	V27__add_relationship_broadcasts.sql

Constraints and later ALTER history:

- CONSTRAINT fk_relationship_broadcast_sender FOREIGN KEY (sender_user_id) REFERENCES users(id)
- CONSTRAINT fk_relationship_broadcast_anchor FOREIGN KEY (anchor_user_id) REFERENCES users(id)
- CONSTRAINT chk_relationship_broadcast_audience CHECK (audience_type IN ('HORIZONTAL','VERTICAL','LOCATION'))

relationships

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V1__create_core_tables.sql
type	VARCHAR(255) NOT NULL	V1__create_core_tables.sql
owner_user_id	BIGINT	V11__add_social_network.sql
related_user_id	BIGINT	V11__add_social_network.sql
contact_name	VARCHAR(255)	V13__add_relationship_display_name.sql
visibility_scope	VARCHAR(20) NOT NULL DEFAULT 'FRIENDS'	V16__add_relationship_visibility.sql
visibility_company	VARCHAR(255)	V16__add_relationship_visibility.sql
contact_phone	VARCHAR(40)	V17__add_private_relationship_contacts.sql
contact_email	VARCHAR(255)	V17__add_private_relationship_contacts.sql
relative_to_user_id	BIGINT	V20__add_relationship_anchor.sql
milestone_date	VARCHAR(10)	V40__add_relationship_milestone_dates.sql
related_birth_date	VARCHAR(10)	V40__add_relationship_milestone_dates.sql
related_death_date	VARCHAR(10)	V40__add_relationship_milestone_dates.sql

Constraints and later ALTER history:

- V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_relationships_type_not_blank CHECK (CHAR_LENGTH(TRIM(type)) > 0)
- V11__add_social_network.sql: ADD CONSTRAINT fk_relationship_owner FOREIGN KEY (owner_user_id) REFERENCES users(id) ON DELETE CASCADE
- V11__add_social_network.sql: ADD CONSTRAINT fk_relationship_related FOREIGN KEY (related_user_id) REFERENCES users(id) ON DELETE CASCADE
- V11__add_social_network.sql: ADD CONSTRAINT uq_relationship_pair UNIQUE (owner_user_id, related_user_id)
- V20__add_relationship_anchor.sql: ADD CONSTRAINT fk_relationship_relative_to FOREIGN KEY (relative_to_user_id) REFERENCES users(id)
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN contact_name TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN contact_phone TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN contact_email TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN milestone_date TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN related_birth_date TYPE TEXT
- V56__encrypted_sensitive_fields.sql: ALTER COLUMN related_death_date TYPE TEXT

role_model_follows

Created by V44__add_trusted_people.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V44__add_trusted_people.sql
`follower_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`role_model_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V44__add_trusted_people.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V44__add_trusted_people.sql

Constraints and later ALTER history:

- `CONSTRAINT uq_role_model_follow UNIQUE (follower_user_id, role_model_user_id)`
- `CONSTRAINT chk_role_model_follow_distinct CHECK (follower_user_id <> role_model_user_id)`

scheduled_job_locks

Created by V53__scheduled_job_locks.sql.

Column	Declaration (with later additions)	Source
`name`	`VARCHAR(80) PRIMARY KEY`	V53__scheduled_job_locks.sql

security_incidents

Created by V66__security_incidents.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V66__security_incidents.sql
`occurred_at`	`TIMESTAMP WITH TIME ZONE NOT NULL`	V66__security_incidents.sql
`category`	`VARCHAR(40) NOT NULL`	V66__security_incidents.sql
`summary`	`VARCHAR(300) NOT NULL`	V66__security_incidents.sql
`request_path`	`VARCHAR(300) NOT NULL`	V66__security_incidents.sql
`ip_address`	`VARCHAR(64) NOT NULL`	V66__security_incidents.sql
`actor`	`VARCHAR(80)`	V66__security_incidents.sql
`blocked_until`	`TIMESTAMP WITH TIME ZONE`	V66__security_incidents.sql
`resolved_at`	`TIMESTAMP WITH TIME ZONE`	V66__security_incidents.sql
`email_status`	`VARCHAR(24) NOT NULL DEFAULT 'PENDING'`	V66__security_incidents.sql
`email_attempts`	`INTEGER NOT NULL DEFAULT 0`	V66__security_incidents.sql
`email_next_attempt`	`TIMESTAMP WITH TIME ZONE NOT NULL`	V66__security_incidents.sql

social_comments

Created by V29__add_social_feed.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V29__add_social_feed.sql
`post_id`	`BIGINT NOT NULL REFERENCES social_posts(id) ON DELETE CASCADE`	V29__add_social_feed.sql
`author_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V29__add_social_feed.sql
`message`	`VARCHAR(1000) NOT NULL`	V29__add_social_feed.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V29__add_social_feed.sql

social_post_likes

Created by V29__add_social_feed.sql.

Column	Declaration (with later additions)	Source
`post_id`	`BIGINT NOT NULL REFERENCES social_posts(id) ON DELETE CASCADE`	V29__add_social_feed.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V29__add_social_feed.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V29__add_social_feed.sql

Constraints and later ALTER history:

- `PRIMARY KEY (post_id, user_id)`

social_post_saves

Created by V37__add_social_post_saves.sql.

Column	Declaration (with later additions)	Source
`post_id`	`BIGINT NOT NULL REFERENCES social_posts(id) ON DELETE CASCADE`	V37__add_social_post_saves.sql
`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V37__add_social_post_saves.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V37__add_social_post_saves.sql

Constraints and later ALTER history:

- `PRIMARY KEY (post_id, user_id)`

social_posts

Created by V29__add_social_feed.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V29__add_social_feed.sql
`author_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V29__add_social_feed.sql
`caption`	`VARCHAR(4000) NOT NULL DEFAULT ''`	V29__add_social_feed.sql
`audience`	`VARCHAR(24) NOT NULL DEFAULT 'RELATIONSHIPS'`	V29__add_social_feed.sql

`circle_id`	`BIGINT REFERENCES circles(id) ON DELETE CASCADE`	V29__add_social_feed.sql
`media_key`	`VARCHAR(255)`	V29__add_social_feed.sql
`media_name`	`VARCHAR(255)`	V29__add_social_feed.sql
`media_type`	`VARCHAR(255)`	V29__add_social_feed.sql
`media_size`	`BIGINT`	V29__add_social_feed.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V29__add_social_feed.sql
`updated_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V29__add_social_feed.sql

Constraints and later ALTER history:

- `CONSTRAINT chk_social_post_audience CHECK (audience IN ('PUBLIC','RELATIONSHIPS','CIRCLE'))`
- `CONSTRAINT chk_social_post_circle CHECK ((audience = 'CIRCLE' AND circle_id IS NOT NULL) OR (audience <> 'CIRCLE' AND circle_id IS NULL))`
- `V43__expand_social_post_audiences.sql: ADD CONSTRAINT chk_social_post_audience CHECK (audience IN ('PRIVATE','PUBLIC','FRIENDS','RELATIVES','RELATIONSHIPS','CIRCLE'))`

social_stories

Created by V29__add_social_feed.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V29__add_social_feed.sql
`author_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V29__add_social_feed.sql
`caption`	`VARCHAR(1000) NOT NULL DEFAULT ''`	V29__add_social_feed.sql
`audience`	`VARCHAR(24) NOT NULL DEFAULT 'RELATIONSHIPS'`	V29__add_social_feed.sql
`media_key`	`VARCHAR(255) NOT NULL`	V29__add_social_feed.sql
`media_name`	`VARCHAR(255)`	V29__add_social_feed.sql
`media_type`	`VARCHAR(255) NOT NULL`	V29__add_social_feed.sql
`media_size`	`BIGINT`	V29__add_social_feed.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V29__add_social_feed.sql
`expires_at`	`TIMESTAMP WITH TIME ZONE NOT NULL`	V29__add_social_feed.sql

Constraints and later ALTER history:

- `CONSTRAINT chk_social_story_audience CHECK (audience IN ('PUBLIC','RELATIONSHIPS'))`

social_story_views

Created by V38__add_social_story_views.sql.

Column	Declaration (with later additions)	Source
`story_id`	`BIGINT NOT NULL REFERENCES social_stories(id) ON DELETE CASCADE`	V38__add_social_story_views.sql

`user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V38__add_social_story_views.sql
`viewed_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V38__add_social_story_views.sql

Constraints and later ALTER history:

- `PRIMARY KEY (story_id, user_id)`

task_groups

Created by V9__add_task_groups.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V9__add_task_groups.sql
`name`	`VARCHAR(255) NOT NULL UNIQUE`	V9__add_task_groups.sql
`description`	`VARCHAR(1024) NOT NULL`	V9__add_task_groups.sql

tasks

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V1__create_core_tables.sql
`title`	`VARCHAR(255) NOT NULL`	V1__create_core_tables.sql
`details`	`VARCHAR(1024) NOT NULL`	V1__create_core_tables.sql
`status`	`VARCHAR(255) NOT NULL`	V1__create_core_tables.sql
`project_id`	`BIGINT`	V1__create_core_tables.sql
`milestone_id`	`BIGINT`	V1__create_core_tables.sql
`task_group_id`	`BIGINT`	V9__add_task_groups.sql

Constraints and later ALTER history:

- `V3__add_indexes_and_foreign_keys.sql: ADD CONSTRAINT fk_tasks_project FOREIGN KEY (project_id) REFERENCES projects(id) ON DELETE SET NULL`
- `V3__add_indexes_and_foreign_keys.sql: ADD CONSTRAINT fk_tasks_milestone FOREIGN KEY (milestone_id) REFERENCES milestones(id) ON DELETE SET NULL`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_tasks_title_not_blank CHECK (CHAR_LENGTH(TRIM(title)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_tasks_details_not_blank CHECK (CHAR_LENGTH(TRIM(details)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_tasks_status_allowed CHECK (status IN ('Todo', 'In Progress', 'Done'))`
- `V9__add_task_groups.sql: ADD CONSTRAINT fk_tasks_task_group FOREIGN KEY (task_group_id) REFERENCES task_groups(id) ON DELETE SET NULL`

timeline_events

Created by V68__timeline_events.sql.

Column	Declaration (with later additions)	Source
user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V68__timeline_events.sql
event_key	VARCHAR(100) NOT NULL	V68__timeline_events.sql
event_date	VARCHAR(10) NOT NULL	V68__timeline_events.sql
kind	VARCHAR(30) NOT NULL	V68__timeline_events.sql
title	VARCHAR(200) NOT NULL	V68__timeline_events.sql
detail	VARCHAR(2000) NOT NULL	V68__timeline_events.sql

Constraints and later ALTER history:

- PRIMARY KEY (user_id, event_key)

trusted_people

Created by V44__add_trusted_people.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V44__add_trusted_people.sql
owner_user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V44__add_trusted_people.sql
trusted_user_id	BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE	V44__add_trusted_people.sql
kind	VARCHAR(20) NOT NULL CHECK (kind IN ('STAR','ROLE_MODEL'))	V44__add_trusted_people.sql
created_at	TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP	V44__add_trusted_people.sql

Constraints and later ALTER history:

- CONSTRAINT uq_trusted_people UNIQUE (owner_user_id, trusted_user_id, kind)
- CONSTRAINT chk_trusted_people_distinct CHECK (owner_user_id <> trusted_user_id)

upload_levels

Created by V67__upload_levels.sql.

Column	Declaration (with later additions)	Source
id	INTEGER PRIMARY KEY CHECK (id BETWEEN 1 AND 5)	V67__upload_levels.sql
name	VARCHAR(50) NOT NULL	V67__upload_levels.sql
storage_bytes	BIGINT NOT NULL CHECK (storage_bytes >= 0)	V67__upload_levels.sql
upload_bytes	BIGINT NOT NULL CHECK (upload_bytes BETWEEN 1 AND 26214400)	V67__upload_levels.sql
max_retention_days	INTEGER NOT NULL CHECK (max_retention_days BETWEEN 1 AND 36500)	V67__upload_levels.sql

`allow_never`	`BOOLEAN NOT NULL`	V67__upload_levels.sql

user_blocks

Created by V31__add_user_blocks.sql.

Column	Declaration (with later additions)	Source
`id`	`BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY`	V31__add_user_blocks.sql
`blocker_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V31__add_user_blocks.sql
`blocked_user_id`	`BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE`	V31__add_user_blocks.sql
`created_at`	`TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP`	V31__add_user_blocks.sql

Constraints and later ALTER history:

- `CONSTRAINT uq_user_block UNIQUE(blocker_user_id,blocked_user_id)`
- `CONSTRAINT chk_user_block_self CHECK(blocker_user_id<>blocked_user_id)`

user_profile_photos

Created by V15__add_user_profiles.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT NOT NULL`	V15__add_user_profiles.sql
`photo_order`	`INTEGER NOT NULL`	V15__add_user_profiles.sql
`photo_data`	`TEXT NOT NULL`	V15__add_user_profiles.sql

Constraints and later ALTER history:

- `PRIMARY KEY (user_id, photo_order)`
- `CONSTRAINT fk_user_profile_photo_user FOREIGN KEY (user_id) REFERENCES user_profiles(user_id) ON DELETE CASCADE`

user_profiles

Created by V15__add_user_profiles.sql.

Column	Declaration (with later additions)	Source
`user_id`	`BIGINT PRIMARY KEY`	V15__add_user_profiles.sql
`date_of_birth`	`VARCHAR(20)`	V15__add_user_profiles.sql
`gender`	`VARCHAR(40)`	V15__add_user_profiles.sql
`bio`	`TEXT`	V15__add_user_profiles.sql
`address_line1`	`VARCHAR(255)`	V15__add_user_profiles.sql

address_line2	VARCHAR(255)	V15__add_user_profiles.sql
city	VARCHAR(120)	V15__add_user_profiles.sql
state	VARCHAR(120)	V15__add_user_profiles.sql
postal_code	VARCHAR(30)	V15__add_user_profiles.sql
country	VARCHAR(120)	V15__add_user_profiles.sql
alternate_phone	VARCHAR(40)	V15__add_user_profiles.sql
website	VARCHAR(255)	V15__add_user_profiles.sql
whatsapp	VARCHAR(120)	V15__add_user_profiles.sql
linkedin	VARCHAR(255)	V15__add_user_profiles.sql
facebook	VARCHAR(255)	V15__add_user_profiles.sql
instagram	VARCHAR(255)	V15__add_user_profiles.sql
x_handle	VARCHAR(120)	V15__add_user_profiles.sql
highest_qualification	VARCHAR(180)	V15__add_user_profiles.sql
institution	VARCHAR(255)	V15__add_user_profiles.sql
field_of_study	VARCHAR(180)	V15__add_user_profiles.sql
graduation_year	VARCHAR(10)	V15__add_user_profiles.sql
employment_status	VARCHAR(80)	V15__add_user_profiles.sql
employer	VARCHAR(255)	V15__add_user_profiles.sql
job_title	VARCHAR(180)	V15__add_user_profiles.sql
industry	VARCHAR(180)	V15__add_user_profiles.sql
work_location	VARCHAR(180)	V15__add_user_profiles.sql
profile_photo	TEXT	V15__add_user_profiles.sql
hobbies	TEXT	V42__add_profile_hobbies.sql

Constraints and later ALTER history:

- `CONSTRAINT fk_user_profile_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE`

users

Created by V1__create_core_tables.sql.

Column	Declaration (with later additions)	Source
id	BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY	V1__create_core_tables.sql
username	VARCHAR(255) NOT NULL UNIQUE	V1__create_core_tables.sql
email	VARCHAR(255) NOT NULL	V1__create_core_tables.sql
password_hash	VARCHAR(255) NOT NULL	V1__create_core_tables.sql
phone_number	VARCHAR(32)	V7__add_user_phone_identity.sql

`role`	`VARCHAR(32) NOT NULL DEFAULT 'USER'`	V8__add_user_roles.sql
`first_name`	`VARCHAR(255)`	V11__add_social_network.sql
`surname`	`VARCHAR(255)`	V11__add_social_network.sql
`location`	`VARCHAR(255)`	V11__add_social_network.sql
`account_status`	`VARCHAR(32) NOT NULL DEFAULT 'ACTIVE'`	V12__add_invited_user_status.sql
`identity_type`	`VARCHAR(24) NOT NULL DEFAULT 'ACCOUNT'`	V19__add_managed_people.sql
`managed_category`	`VARCHAR(24)`	V19__add_managed_people.sql
`guardian_user_id`	`BIGINT`	V19__add_managed_people.sql
`claim_status`	`VARCHAR(48) NOT NULL DEFAULT 'NONE'`	V19__add_managed_people.sql
`managed_date_of_birth`	`VARCHAR(10)`	V19__add_managed_people.sql
`managed_date_of_death`	`VARCHAR(10)`	V19__add_managed_people.sql
`managed_notes`	`VARCHAR(2000)`	V19__add_managed_people.sql
`phone_verified`	`BOOLEAN NOT NULL DEFAULT FALSE`	V55__verified_phone_and_admin_mfa.sql
`upload_level_id`	`INTEGER NOT NULL DEFAULT 1 REFERENCES upload_levels(id)`	V67__upload_levels.sql

Constraints and later ALTER history:

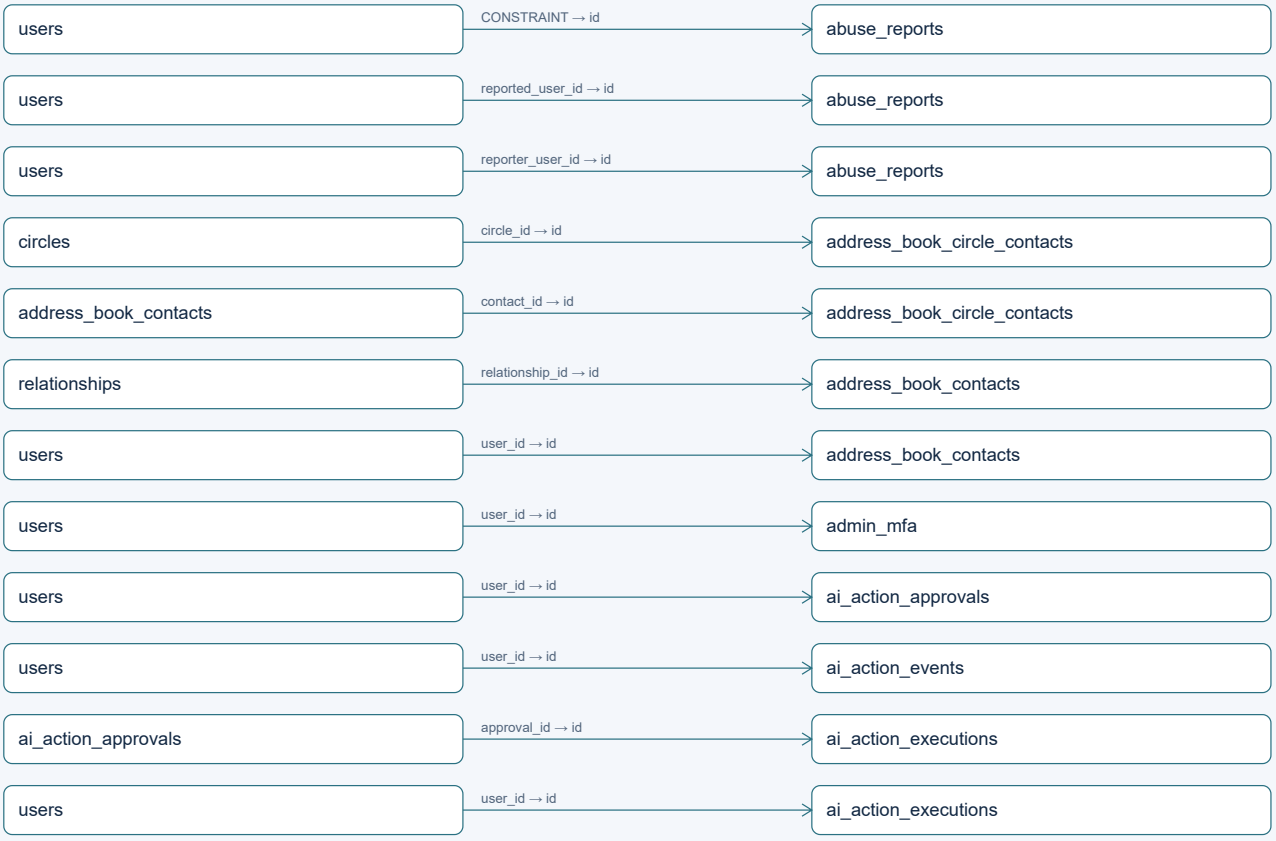
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_users_username_not_blank CHECK (CHAR_LENGTH(TRIM(username)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_users_email_not_blank CHECK (CHAR_LENGTH(TRIM(email)) > 0)`
- `V4__add_data_quality_constraints.sql: ADD CONSTRAINT chk_users_password_hash_not_blank CHECK (CHAR_LENGTH(TRIM(password_hash)) > 0)`
- `V6__add_unique_user_email.sql: ADD CONSTRAINT uq_users_email UNIQUE (email)`
- `V7__add_user_phone_identity.sql: ALTER COLUMN phone_number SET NOT NULL`
- `V7__add_user_phone_identity.sql: ADD CONSTRAINT uq_users_phone_number UNIQUE (phone_number)`
- `V7__add_user_phone_identity.sql: ALTER COLUMN email DROP NOT NULL`
- `V19__add_managed_people.sql: ALTER COLUMN phone_number DROP NOT NULL`
- `V19__add_managed_people.sql: ADD CONSTRAINT fk_users_guardian FOREIGN KEY (guardian_user_id) REFERENCES users(id)`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN managed_notes TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN managed_date_of_birth TYPE TEXT`
- `V56__encrypted_sensitive_fields.sql: ALTER COLUMN managed_date_of_death TYPE TEXT`

Appendix C — Physical relationship map

Explicit foreign-key references; see editable Mermaid files for domain diagrams. Ownership without an FK remains a service rule.

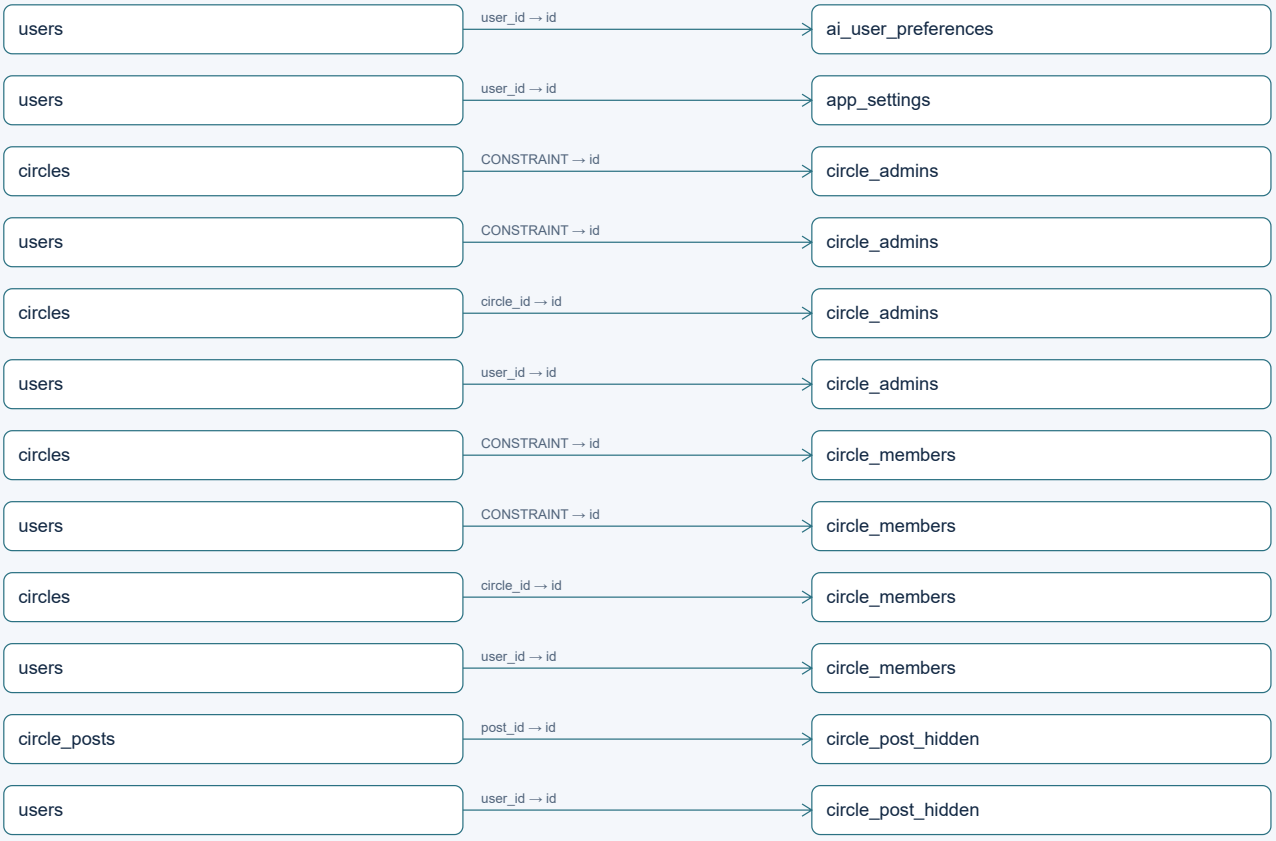
Foreign-key map — sheet 1

Foreign-key relationships 1–12



Foreign-key map — sheet 2

Foreign-key relationships 13–24



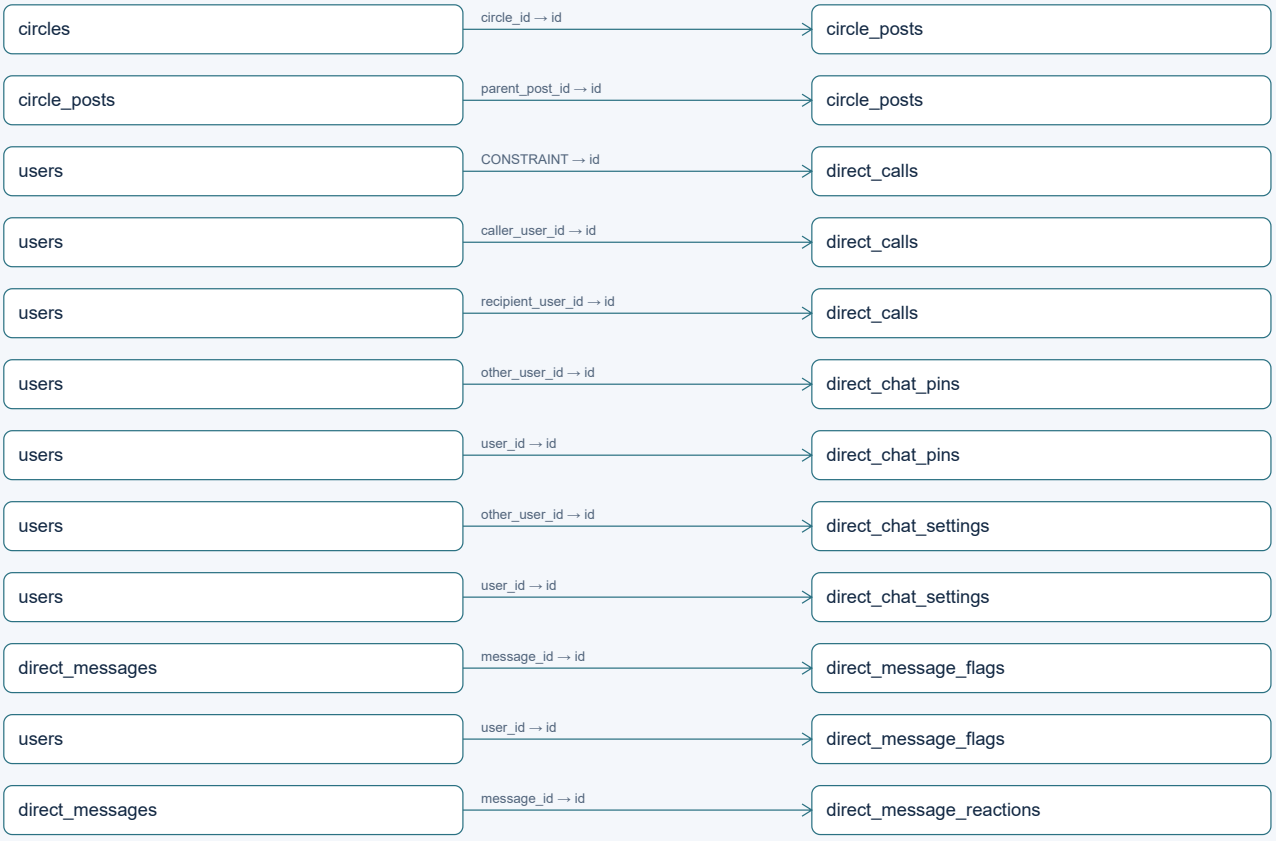
Foreign-key map — sheet 3

Foreign-key relationships 25–36



Foreign-key map — sheet 4

Foreign-key relationships 37–48



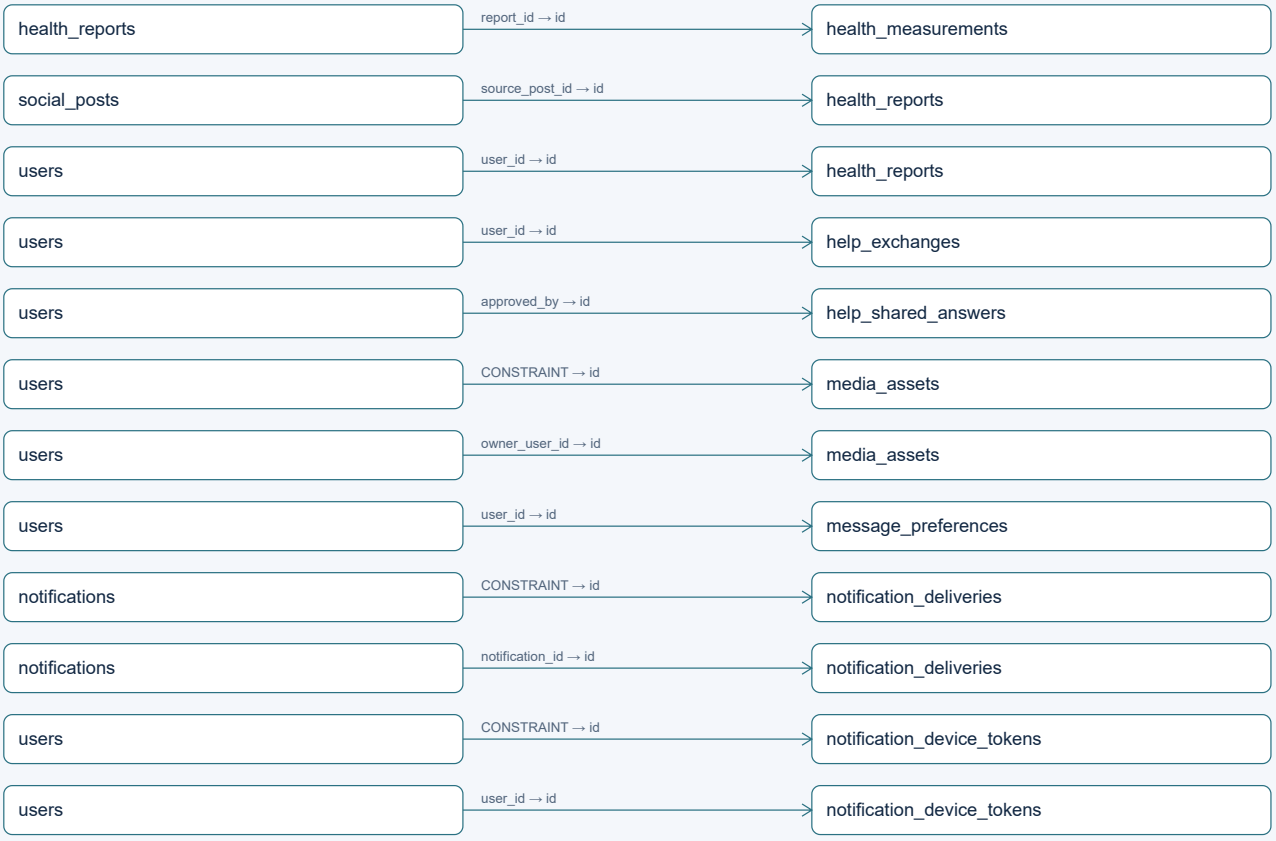
Foreign-key map — sheet 5

Foreign-key relationships 49–60



Foreign-key map — sheet 6

Foreign-key relationships 61–72



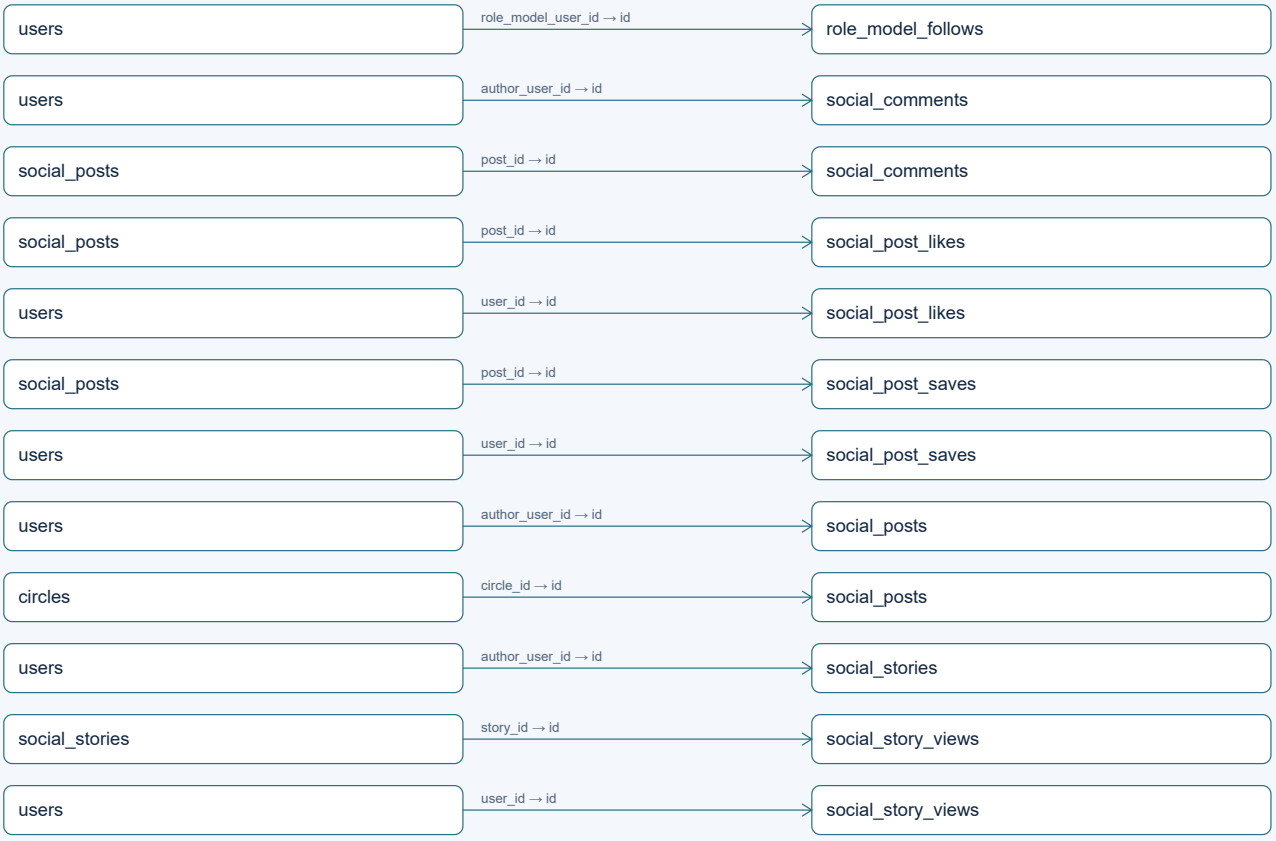
Foreign-key map — sheet 7

Foreign-key relationships 73–84



Foreign-key map — sheet 8

Foreign-key relationships 85–96



Foreign-key map — sheet 9

Foreign-key relationships 97–106

